RAG TECHNIQUES CATALOG

Retrieval-Augmented Generation

40 Tecniche Essenziali

Una scheda per tecnica con architettura, implementazione,
pro/contro, complessità, **esempio di codice Python** e riferimenti bibliografici.

40 tecniche suddivise in 8 categorie:
Foundational · Query Enhancement · Context Enrichment
Advanced Retrieval · Iterative · Advanced Architecture · Evaluation · Explainability

•

Francesco Collovà
francesco.collova @ gmail.com https://www.linkedin.com/in/fcollova/

Versione 1.0 • 2026 • Con esempi di codice Python

Indice delle Tecniche

Foundational (1–6)

1. Basic RAG
2. Reliable RAG
3. Optimizing Chunk Sizes
4. Proposition Chunking
5. Document Augmentation
6. RAG with CSV / Structured Data

Query Enhancement (7–12)

7. Query Transformations
8. HyDE
9. HyPE
10. Step-Back Prompting
11. Multi-Query Retrieval
12. Query Routing

Context Enrichment (13–19)

13. Contextual Chunk Headers
14. Semantic Chunking
15. Context Window Enhancement
16. Relevant Segment Extraction
17. Contextual Compression
18. Document Hierarchy (Parent-Child)
19. Late Chunking

Advanced Retrieval (20–27)

20. Fusion Retrieval
21. Reranking
22. Multi-faceted Filtering
23. Hierarchical Indices
24. Ensemble Retrieval
25. Dartboard Retrieval (MMR)
26. Multi-modal RAG
27. ColBERT / Token-level Retrieval

Iterative (28–31)

28. Iterative Retrieval
29. Adaptive Retrieval
30. Retrieval with Feedback Loop
31. FLARE

Advanced Architecture (32–38)

32. Agentic RAG
33. Self-RAG
34. Corrective RAG (CRAG)
35. Graph RAG
36. Microsoft GraphRAG
37. RAPTOR
38. Sophisticated Controllable Agent

Evaluation (39)

39. DeepEval

Explainability (40)

40. Explainable Retrieval

Ogni scheda include: Descrizione estesa • Diagramma architetturale • Pro/Contro • Complessità • **Esempio di codice Python** • Riferimenti bibliografici

#1 Basic RAG

Foundational Standard RAG

Key feature: **Top-k retrieval + prompt augmentation**

DESCRIZIONE

Il Basic RAG è il punto di partenza di ogni sistema RAG. La query viene trasformata in embedding, si esegue una ricerca vettoriale, i top-k chunk vengono iniettati nel prompt come contesto e l'LLM genera la risposta condizionata a quell'evidenza. Pur nella sua semplicità, questo approccio migliora significativamente la fattualità rispetto a un LLM puro, riducendo le allucinazioni su domini specifici. È il baseline obbligatorio prima di introdurre tecniche più complesse.

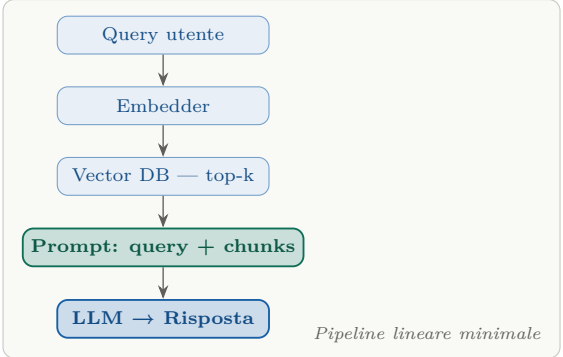
COME FUNZIONA

Indicizza il corpus con embedding. A runtime embeds la query, cerca i top-k chunk più simili nel vector DB, costruisce il prompt inserendo i chunk come contesto, genera la risposta. Il modello di embedding deve essere lo stesso usato a index-time.

PASSI IMPLEMENTATIVI

1. Chunking del corpus (size 512–800 token, overlap 10%)
2. Embedding di ogni chunk e upsert nel vector DB
3. A runtime: embed query con lo stesso modello
4. Vector search cosine similarity, top-k (3–8)
5. Costruisci prompt: istruzione + contesto + query
6. Genera risposta; estrai citazioni (opzionale)

DIAGRAMMA ARCHITETTURALE



PRO

- + Veloce da costruire, ottimo baseline
- + Migliora la fattualità rispetto a LLM puro
- + Pipeline semplice e debuggabile

CONTRO

- – Sensibile alla qualità del chunking
- – Può perdere evidenze chiave
- – Allucinazioni se il contesto è debole

TOOL E LIBRERIE

LangChain / LlamaIndex • FAISS, Chroma, pgvector • text-embedding-3-small, e5-large, bge-m3

COMPLESSITÀ

★☆☆ Bassa

USE CASE

Q&A su corpus fisso, FAQ, documentazione tecnica, knowledge base aziendali

ESEMPIO PYTHON

```
pip install langchain, faiss-cpu, openai
```

```python
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

embeddings = OpenAIEmbeddings()
vectordb   = FAISS.from_documents(docs, embeddings)
retriever  = vectordb.as_retriever(search_kwargs={"k": 5})

qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(), retriever=retriever,
    chain_type="stuff"
)
answer = qa_chain.run("Qual e' il limite di dosaggio?")
```

RIFERIMENTI BIBLIOGRAFICI

- Lewis et al., *RAG for NLP* (NeurIPS 2020). arxiv:2005.11401
- Gao et al., *RAG Survey* (2024). arxiv:2312.10997
- LangChain RAG docs. python.langchain.com

#2 Reliable RAG

Foundational Reliability / Grounding

Key feature: **Validation loop: relevance + grounding check**

DESCRIZIONE

Reliable RAG aggiunge un doppio layer di validazione al pipeline base. Prima verifica se il contesto recuperato è rilevante per la query; poi, dopo la generazione, controlla se la risposta è grounded nel contesto. In caso negativo il sistema riscrive la query e ri-recupera. Questo pattern riduce le allucinazioni in scenari ad alto rischio senza stravolgere l'architettura base.

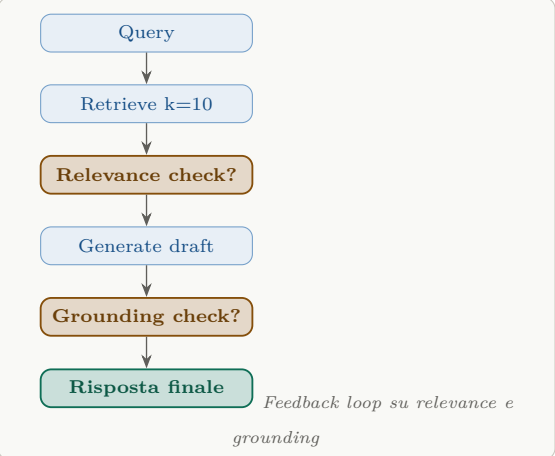
COME FUNZIONA

Dopo ogni retrieve si chiede a un LLM giudice (yes/no) se il contesto è rilevante. Se no, si riscrive la query. Dopo la generazione si chiede se la risposta è supportata dal contesto. Se no, si raffina il draft. Massimo 2 iterazioni per evitare loop.

PASSI IMPLEMENTATIVI

1. Retrieve $k = 10$ chunk
2. Relevance check: LLM yes/no sul contesto
3. Se no: rewrite query e re-retrieve $k = 15$
4. Genera draft con contesto validato
5. Grounding check: risposta supportata dal contesto?
6. Se no: refine prompt con aderenza esplicita

DIAGRAMMA ARCHITETTURALE



PRO

- + Riduce le allucinazioni su domini critici
- + Più tracciabile e auditabile
- + Compatibile con pipeline esistenti

CONTRO

- – Latenza aumenta di 1–2 LLM call
- – Validatori imperfetti (falsi negativi)
- – Richiede soglie e test set per calibrare

TOOL E LIBRERIE

LLM-as-judge (GPT-3.5/4) • DeepEval, TruLens • LangSmith per tracing

COMPLESSITÀ ★★☆ Media

USE CASE

Domini ad alto rischio: legale, medicale, finance, support automation

ESEMPIO PYTHON

```
pip install deepeval, langchain
```

```python
from deepeval.metrics import FaithfulnessMetric, AnswerRelevancyMetric
from deepeval import evaluate

# Relevance check: context vs query
rel_metric  = AnswerRelevancyMetric(threshold=0.7)
faith_metric = FaithfulnessMetric(threshold=0.7)

# Run RAG + validate
context, answer = rag_pipeline(query)
test_case = LLMTestCase(input=query,
    actual_output=answer, retrieval_context=context)
evaluate([test_case], [rel_metric, faith_metric])
# Se fallisce -> rewrite query e re-retrieve
```

RIFERIMENTI BIBLIOGRAFICI

- Gao et al., *RAG Survey* (2024). arxiv:2312.10997
- Saad-Falcon et al., *ARES* (2023). arxiv:2311.09476
- TruLens docs. trulens.org

#3 Optimizing Chunk Sizes

Foundational Chunking / Indexing

Key feature: **Chunk & overlap tuning con offline evaluation**

DESCRIZIONE

Il chunking è uno dei parametri più impattanti ma spesso sottovalutati. Chunk troppo piccoli frammentano il contesto; chunk troppo grandi portano rumore. Il tuning si effettua offline su un test set rappresentativo, misurando retrieval hit rate, faithfulness e relevancy per ogni configurazione di size e overlap. L'overlap (10–15%) aiuta a non spezzare frasi a metà.

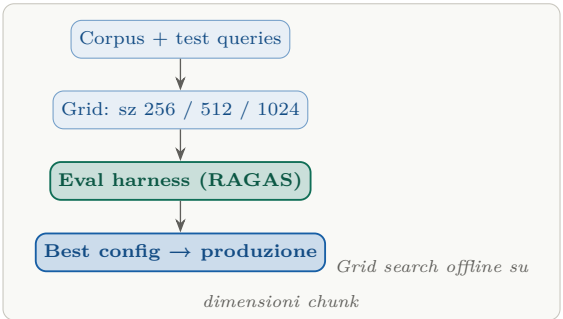
COME FUNZIONA

Si costruisce una griglia di configurazioni (dimensione chunk, overlap) e si valuta ciascuna su un golden dataset. La metrica principale è il retrieval hit rate combinato con faithfulness. La configurazione migliore viene usata per re-indicizzare il corpus in produzione.

PASSI IMPLEMENTATIVI

1. Costruisci golden dataset di query con risposta attesa
2. Definisci griglia: size in [256,512,800,1200], overlap in [0,5%,10%,15%]
3. Per ogni config: chunking + index + eval su golden set
4. Misura: hit rate, faithfulness, token cost
5. Seleziona config con miglior trade-off precision/recall
6. Re-indicizza il corpus con la config ottimale

DIAGRAMMA ARCHITETTURALE



PRO

- + Alto ROI con sforzo moderato
- + Migliora recall e coerenza
- + Riduce token sprecati nel context window

CONTRO

- – Dipende dai dati (non universale)
- – Richiede re-indicizzazione ad ogni cambio
- – Rischio di overfitting sul test set

TOOL E LIBRERIE

RAGAS per evaluation • DeepEval • LlamaIndex / LangChain chunkers

COMPLESSITÀ ★☆☆ Bassa

USE CASE

Qualsiasi sistema RAG — da eseguire prima di introdurre complessità

ESEMPIO PYTHON

```
pip install ragas, langchain
```

```python
from ragas import evaluate
from ragas.metrics import (faithfulness,
    answer_relevancy, context_precision)

# Build eval dataset
dataset = Dataset.from_dict({
    "question": questions, "answer": answers,
    "contexts": contexts, "ground_truth": gts
})
# Score ogni configurazione di chunking
scores = evaluate(dataset, metrics=[
    faithfulness, answer_relevancy, context_precision
])
print(scores)  # -> scegli config con score migliore
```

RIFERIMENTI BIBLIOGRAFICI

- Es et al., *RAGAS* (2023). arxiv:2309.15217
- Kamradt, *Chunking strategies* (2024). github

#4 Proposition Chunking

Foundational Chunking / Knowledge extraction

Key feature: **LLM-assisted atomic proposition extraction**

DESCRIZIONE

Invece di dividere il testo in chunk di dimensione fissa, si usano proposizioni fattuali atomiche estratte via LLM come unità di indicizzazione. Ogni proposizione è una singola affermazione verificabile ('La CPU ha 8 core'). Questo approccio massimizza la precisione del retrieval per domande fattuali, perché ogni unità recuperata contiene esattamente un fatto rilevante.

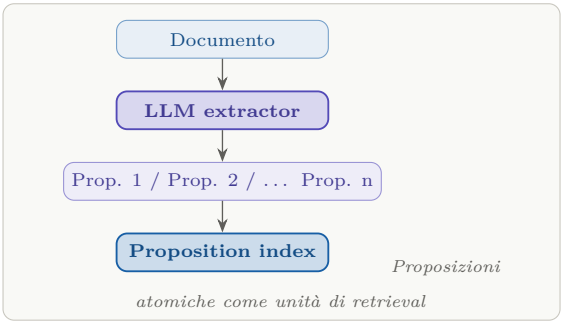
COME FUNZIONA

A index-time, ogni documento viene passato a un LLM che estrae una lista di proposizioni atomiche. Ogni proposizione viene indicizzata separatamente con un riferimento al documento sorgente. A retrieval-time si cercano le proposizioni più rilevanti. La risposta è quindi grounded su affermazioni precise.

PASSI IMPLEMENTATIVI

1. Passa ogni documento a un LLM extractor
2. Ottieni lista di proposizioni atomiche ('X è Y', 'A fa B')
3. Index ogni proposizione con metadata {doc_id, page}
4. Retrieval: cerca top-k proposizioni per la query
5. Raggruppa proposizioni per documento sorgente
6. Genera risposta citando le proposizioni di supporto

DIAGRAMMA ARCHITETTURALE



PRO

- + Alta precisione per Q&A fattuali
- + Grounding fine-grained, citazioni precise
- + Ottimo per policy, spec tecniche, compliance

CONTRO

- – Costo elevato a index-time (LLM call per ogni doc)
- – La qualità dipende dall'extractor LLM
- – Può perdere il flusso narrativo del testo

TOOL E LIBRERIE

LLM extractor (GPT-4, Claude) • Vector DB con metadata

- LlamaIndex proposition extractor

COMPLESSITÀ ★★☆ Media

USE CASE

Compliance, policy aziendali, specifiche tecniche, fact lookup su knowledge base

ESEMPIO PYTHON

```
pip install llama-index
```

```python
from llama_index.core.node_parser import (
    SentenceSplitter, SimpleNodeParser)
from llama_index.core.extractors import (
    QuestionsAnsweredExtractor)
from llama_index.core.ingestion import IngestionPipeline

pipeline = IngestionPipeline(transformations=[
    SentenceSplitter(chunk_size=512),
    QuestionsAnsweredExtractor(  # estrae proposizioni
        questions=5, llm=llm
    ),
    OpenAIEmbedding(),
])
nodes = pipeline.run(documents=docs)
index  = VectorStoreIndex(nodes)
```

RIFERIMENTI BIBLIOGRAFICI

- Chen et al., *Dense Passage Retrieval* (2020). arxiv:2004.04906
- Min et al., *FActScoring* (2023). arxiv:2305.14251

#5 Document Augmentation

Foundational Synthetic data augmentation

Key feature: **Index-time augmentation con domande sintetiche**

DESCRIZIONE

Arricchisce ogni documento a index-time con domande sintetiche generate da un LLM, aumentando la recuperabilità del documento quando l'utente usa phrasing diverso da quello del testo originale. È l'inverso di HyDE: invece di generare un documento ipotetico dalla query, si generano domande ipotetiche dal documento. Molto utile per corpora sparse con linguaggio tecnico.

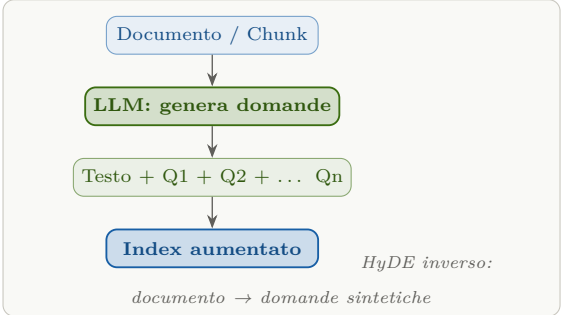
COME FUNZIONA

A index-time, per ogni documento o chunk si chiedono all'LLM 3–5 domande che quel testo risponderebbe. Le domande vengono concatenate al testo originale e indicizzate insieme. A runtime, la query dell'utente troverà il documento anche se usa un vocabolario diverso dal testo originale.

PASSI IMPLEMENTATIVI

1. Per ogni chunk, chiedi all'LLM: 'Genera 5 domande che questo testo risponde'
2. Concatena le domande generate al testo originale
3. Chunka e indicizza il testo aumentato
4. Opzionale: indicizza separatamente domande e testo con link
5. A runtime: retrieval normale sulla query dell'utente
6. Le domande sintetiche aumentano il recall su phrasing diversi

DIAGRAMMA ARCHITETTURALE



PRO

- + Aumenta il recall su phrasing variabili
- + Aiuta query indirette e colloquiali
- + Ottimo per KB sparse con linguaggio tecnico

CONTRO

- – Costo a index-time (LLM call per ogni chunk)
- – Rischio di rumore sintetico se l'LLM genera domande irrilevanti
- – Richiede valutazione per verificare il miglioramento

TOOL E LIBRERIE

LLM question generator • Vector DB • LlamaIndex DocumentSummaryIndex

COMPLESSITÀ ★★☆ Media

USE CASE

Corpora sparse, manualistica tecnica, linguaggio utente variegato rispetto al corpus

ESEMPIO PYTHON

```
pip install llama-index, openai
```

```python
from llama_index.core.indices import (
    DocumentSummaryIndex)

# Augmenta ogni doc con domande sintetiche
index = DocumentSummaryIndex.from_documents(
    documents,
    llm=llm,
    transformations=[SentenceSplitter(chunk_size=1024)],
    show_progress=True,
)
# A runtime: il retrieval trova anche tramite
# le domande sintetiche generate a index-time
query_engine = index.as_query_engine(
    response_mode="tree_summarize")
response = query_engine.query(query)
```

RIFERIMENTI BIBLIOGRAFICI

- Ma et al., *FLARE* (2023). arxiv:2305.06983
- Wang et al., *Query2Doc* (2023). arxiv:2303.07678

#6 RAG with CSV / Structured Data

Foundational Standard RAG (structured source)

Key feature: **Row-wise indexing + metadata filters**

DESCRIZIONE

Estende il RAG a fonti dati strutturate (CSV, tabelle, database) convertendo ogni riga o gruppo di righe in testo recuperabile con metadati. A differenza del RAG su testo libero, ogni unità di retrieval ha una provenienza precisa (row ID, colonne) e può essere filtrata per attributi strutturati prima o dopo la ricerca vettoriale.

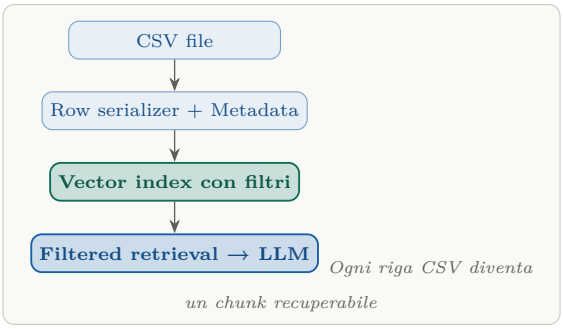
COME FUNZIONA

Ogni riga del CSV viene serializzata in una stringa leggibile ('campo: valore, ...') e indicizzata come un chunk. I metadati strutturati (ID riga, categoria, data) vengono conservati nel vector DB per il filtraggio. A runtime, la query viene arricchita di filtri estratti automaticamente.

PASSI IMPLEMENTATIVI

1. Carica il CSV con Pandas
2. Per ogni riga: serializza in stringa 'campo: valore'
3. Upsert nel vector DB con metadata {row_id, colonne chiave}
4. A runtime: estrai filtri dalla query (NLP o regex)
5. Vector search con where-clause sui metadata
6. Genera risposta citando i row ID sorgente

DIAGRAMMA ARCHITETTURALE



PRO

- + Provenienza precisa per row ID
- + Facile filtro per colonna/categoria
- + Ottimo per cataloghi, export, lookup

CONTRO

- – Schema-to-text design richiede cura
- – Join e relazioni richiedono logica extra
- – CSV molto grandi richiedono raggruppamento o aggregazione

TOOL E LIBRERIE

Pandas + DuckDB • pgvector / Qdrant con metadata • Text-to-SQL come alternativa per query complesse

COMPLESSITÀ ★☆☆ Bassa

USE CASE

Cataloghi prodotti, ticket export, lookup table, report strutturati

ESEMPIO PYTHON

pip install pandas, langchain, faiss-cpu

```python
import pandas as pd
from langchain.docstore.document import Document

df = pd.read_csv("products.csv")

# Serializza ogni riga come testo
docs = [
    Document(
        page_content="\n".join(
            f"{col}: {row[col]}" for col in df.columns),
        metadata={"row_id": i, "category": row["cat"]}
    )
    for i, row in df.iterrows()
]
vectordb  = FAISS.from_documents(docs, embeddings)
retriever = vectordb.as_retriever(
    search_kwargs={"k": 5,
        "filter": {"category": "electronics"}})
```

RIFERIMENTI BIBLIOGRAFICI

- LlamaIndex Structured Data docs. docs.llamaindex.ai
- Pourreza et al., *DIN-SQL* (2023). arxiv:2304.11015

#7 Query Transformations

Query Enhancement Query rewriting / decomposition

Key feature: **Rewrite fedele + decomposizione in sub-query**

DESCRIZIONE

Le query degli utenti sono spesso vaghe, ambigue o troppo complesse per essere soddisfatte da un singolo retrieval. La Query Transformation genera varianti fedeli della query originale e la decompone in sotto-domande più specifiche. I risultati di retrieval di tutte le varianti vengono poi deduplicati e fusi. Questo approccio migliora significativamente il recall senza modificare il corpus.

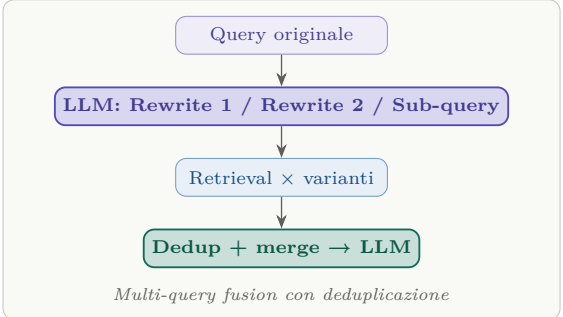
COME FUNZIONA

Un LLM genera 2–3 riformulazioni fedeli della query e, se necessario, la decompone in sotto-domande. Per ogni variante si esegue un retrieval separato. I risultati vengono deduplicati (per ID chunk) e i top-k unici vengono passati all'LLM per la risposta finale alla query originale.

PASSI IMPLEMENTATIVI

1. LLM: genera 2–3 riformulazioni fedeli della query
2. LLM: decomponi in sotto-domande se la query è composita
3. Esegui retrieval per ogni variante/sotto-domanda
4. Deduplica i risultati per chunk ID
5. Seleziona top-8 unici per rilevanza
6. Genera risposta sulla query originale con contesto fuso

DIAGRAMMA ARCHITETTURALE



PRO

+ Migliora il recall su query vaghe
+ Gestisce domande multi-parte
+ Nessuna modifica al corpus necessaria

CONTRO

– Più retrieval call (latenza)
– Può deviare dal significato originale senza constraints
– Richiede guardrail sul rewriter

TOOL E LIBRERIE

LLM rewriter • Hybrid retrievers • Caching delle query frequenti

COMPLESSITÀ ★★☆ Media

USE CASE

Assistenti conversazionali, domande ambigue, Q&A multi-hop

ESEMPIO PYTHON

```
pip install langchain, openai
```

```python
from langchain.retrievers import MultiQueryRetriever
from langchain.chat_models import ChatOpenAI

# Genera riformulazioni + decompone in sub-query
retriever = MultiQueryRetriever.from_llm(
    retriever=vectordb.as_retriever(),
    llm=ChatOpenAI(temperature=0),
    include_original=True  # includi query originale
)
# Esegue retrieval per ogni variante e deduplica
unique_docs = retriever.get_relevant_documents(
    "Come configuro il timeout del server?"
)
print(f"{len(unique_docs)} chunk unici trovati")
```

RIFERIMENTI BIBLIOGRAFICI

- Ma et al., *Query Rewriting for RAG* (2023). arxiv:2305.14283
- Rackauckas, *RAG Fusion* (2024). arxiv:2402.03367

#8 HyDE

Query Enhancement Query-to-document synthesis

Key feature: **Hypothetical Document Embedding**

DESCRIZIONE

HyDE (Hypothetical Document Embedding) affronta il problema del mismatch tra il registro della query (breve, colloquiale) e il registro dei documenti (tecnico, dettagliato). Invece di cercare direttamente con l'embedding della query, genera prima un documento ipotetico che risponderebbe alla query, poi usa l'embedding di quel documento per la ricerca. La distanza semantica tra il documento ipotetico e i documenti reali è molto più piccola della distanza query-documento.

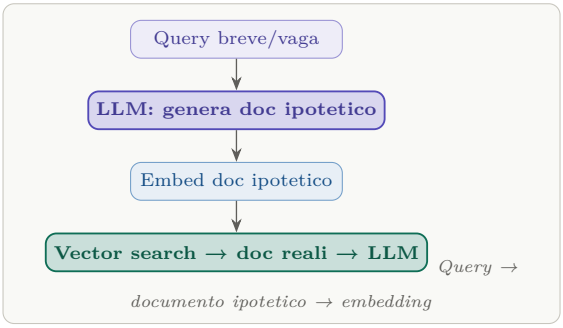
COME FUNZIONA

L'LLM genera un passaggio ipotetico plausibile che risponderebbe alla query. Questo passaggio viene embeddato con lo stesso modello usato per il corpus. La ricerca viene eseguita con questo embedding 'documento-like' invece che con l'embedding della query originale, trovando documenti reali strutturalmente simili al documento ipotetico.

PASSI IMPLEMENTATIVI

1. LLM: genera documento ipotetico (1–2 paragrafi) che risponde alla query
2. Embeds il documento ipotetico con il modello del corpus
3. Vector search con embedding ipotetico (non quello della query)
4. Retrieval dei documenti reali più simili
5. Genera risposta finale basandosi sui documenti reali

DIAGRAMMA ARCHITETTURALE



PRO

- + Migliora retrieval su query corte o vaghe
- + Aiuta le long-tail questions
- + Funziona con qualsiasi vector DB senza modifiche

CONTRO

- – LLM call extra (latenza e costo)
- – Il documento ipotetico può fuorviare se l'LLM allucinazione
- – Richiede prompt attenti per evitare documenti troppo generici

TOOL E LIBRERIE

LLM per generazione ipotetica • Embedder + Vector DB • Cohere, OpenAI embeddings

COMPLESSITÀ ★★☆ Media

USE CASE

Query sparse, ricerca scientifica, domande su domini tecnici specializzati

ESEMPIO PYTHON

```
pip install langchain, openai
```

```python
from langchain.chains import HypotheticalDocumentEmbedder
from langchain.embeddings import OpenAIEmbeddings
from langchain.llms import OpenAI

base_embeddings = OpenAIEmbeddings()
llm = OpenAI()

# Genera documento ipotetico e ne prende l'embedding
embeddings = HypotheticalDocumentEmbedder.from_llm(
    llm=llm, base_embeddings=base_embeddings,
    custom_prompt=hyde_prompt  # template opzionale
)
vectordb  = FAISS.from_documents(docs, embeddings)
retriever = vectordb.as_retriever()
# La ricerca usa l'embedding del doc ipotetico
```

RIFERIMENTI BIBLIOGRAFICI

- Gao et al., *Precise Zero-Shot Dense Retrieval via HyDE* (2022). arxiv:2212.10496

#9 HyPE

Query Enhancement Prompt synthesis for retrieval

Key feature: **Hypothetical Prompt Embedding**

DESCRIZIONE

HyPE (Hypothetical Prompt Embedding) è complementare a HyDE. Mentre HyDE genera documenti ipotetici lato query, HyPE genera istruzioni task-oriented che catturano l'intent della query in forma di prompt, e usa queste istruzioni come embedding per steer il retrieval verso i documenti più adatti al task specifico (summarize, compare, extract, etc.).

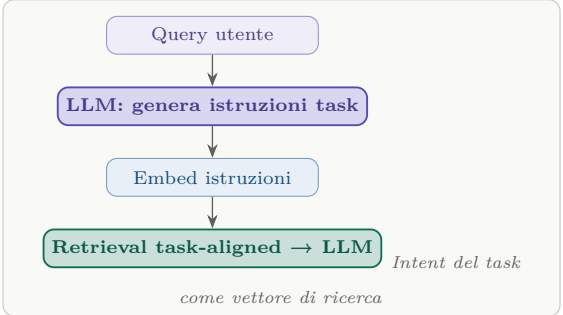
COME FUNZIONA

L'LLM genera istruzioni task-focused che descrivono cosa si sta cercando di fare con la query. Queste istruzioni vengono embeddated e usate per la ricerca. L'effetto è di selezionare documenti che si prestano al task richiesto (es. per una richiesta di confronto, recupera documenti comparativi).

PASSI IMPLEMENTATIVI

1. Analizza l'intent della query (summarize? compare? extract? define?)
2. LLM: genera istruzioni task-focused per la query
3. Embeds le istruzioni come query alternativa
4. Vector search con embedding delle istruzioni
5. Retrieval dei documenti più adatti al task
6. Genera risposta orientata al task con i documenti recuperati

DIAGRAMMA ARCHITETTURALE



PRO

- + Allinea il retrieval all'intent del task
- + Utile per summarize, compare, extract
- + Constraint-friendly: le istruzioni sono controllabili

CONTRO

- – LLM call extra
- – Può over-specificare e perdere documenti rilevanti
- – Richiede tuning per task diversi

TOOL E LIBRERIE

LLM prompt synthesizer • Embedder + Vector DB • Router opzionale per tipo di task

COMPLESSITÀ ★★☆ Media

USE CASE

Assistenti orientati ai task, report automatici, pipeline di analisi documentale

ESEMPIO PYTHON

pip install sentence-transformers, openai

```python
# HyPE: genera istruzioni task-focused
# e le usa come query embedding

prompt = ("Genera istruzioni task-oriented per: {query}\n"
          "Esempio: 'Trovare documentazione tecnica su ...' ")

task_instruction = llm.invoke(
    prompt.format(query=user_query))

# Embeds le istruzioni (non la query raw)
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
query_emb = model.encode(task_instruction)

# Vector search con embedding task-aligned
results = vectordb.similarity_search_by_vector(
    query_emb, k=5)
```

RIFERIMENTI BIBLIOGRAFICI

- Springer et al., *HyPE: Hypothetical Prompt Embeddings* (2025). arxiv:2501.06228

#10 Step-Back Prompting

Query Enhancement Query abstraction

Key feature: **Astrazione della query al livello superiore**

DESCRIZIONE

Step-Back Prompting è una tecnica di query enhancement che chiede all'LLM di fare un passo indietro e riformulare la query a un livello di astrazione più alto prima del retrieval. Invece di cercare direttamente la risposta specifica, si cerca prima il principio generale o il contesto più ampio, migliorando il retrieval di documenti rilevanti che non contengono esplicitamente i termini della query originale.

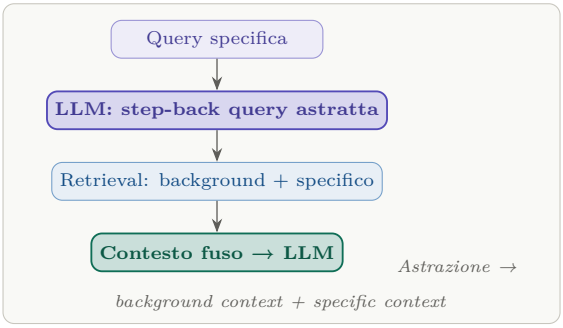
COME FUNZIONA

Prima del retrieval, l'LLM genera una versione 'astratta' della query originale. Ad esempio, 'Quale è il farmaco X usato per Y in pazienti anziani?' diventa 'Principi di farmacologia per Y'. Si esegue il retrieval sia sulla query astratta che su quella originale, poi si fonde il contesto per la risposta finale.

PASSI IMPLEMENTATIVI

1. LLM: astrai la query al livello superiore (principio, categoria, contesto)
2. Retrieval sulla query astratta (background context)
3. Retrieval sulla query originale (specific context)
4. Fuse i due contesti
5. Genera risposta con contesto background + specifico

DIAGRAMMA ARCHITETTURALE



PRO

- + Migliora retrieval su query molto specifiche
- + Recupera il contesto di background necessario
- + Riduce le allucinazioni per mancanza di contesto

CONTRO

- – LLM call extra per l'astrazione
- – Può astrarre troppo e perdere specificità
- – Richiede esempi few-shot per guidare il passo indietro

TOOL E LIBRERIE

LLM con few-shot prompting • Vector DB • LangChain Step-Back chain

COMPLESSITÀ ★★☆ Media

USE CASE

Domande scientifiche, mediche, legali dove la risposta richiede background teorico

ESEMPIO PYTHON

```
pip install langchain, openai
```

```python
from langchain import PromptTemplate, LLMChain

step_back_prompt = PromptTemplate.from_template(
    "Riscrivi questa query in modo piu' astratto, "
    "al livello di principio generale:\n{query}"
)
chain = LLMChain(llm=llm, prompt=step_back_prompt)

abstract_query = chain.run(query=user_query)

# Retrieval su entrambe: astratta + originale
background = retriever.get_relevant_documents(
    abstract_query)
specific   = retriever.get_relevant_documents(
    user_query)
context = background + specific  # fonde i contesti
```

RIFERIMENTI BIBLIOGRAFICI

- Zheng et al., *Take a Step Back: Evoking Reasoning via Abstraction* (2023). arxiv:2310.06117

#11 Multi-Query Retrieval

Query Enhancement Query expansion

Key feature: **Generazione parallela di N query diverse**

DESCRIZIONE

Multi-Query Retrieval genera automaticamente N varianti della query originale da prospettive diverse, esegue retrieval in parallelo per ognuna e fonde i risultati. A differenza della semplice riformulazione, le varianti possono avere angolazioni semantiche diverse (definizione, esempio, confronto), massimizzando la copertura del corpus per query complesse o ambigue.

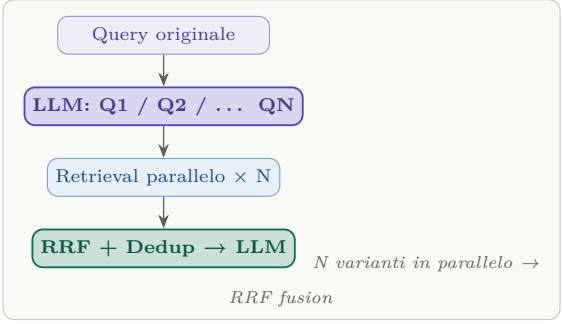
COME FUNZIONA

Un LLM genera N (tipicamente 3–5) versioni della query da angolazioni diverse. Ogni versione viene usata per un retrieval indipendente. I risultati vengono aggregati con Reciprocal Rank Fusion o union semplice, deduplicati e i migliori passati all'LLM. Il sistema può essere parallelizzato facilmente.

PASSI IMPLEMENTATIVI

1. LLM: genera N varianti della query da prospettive diverse
2. Esegui retrieval in parallelo per ogni variante
3. Raccogli tutti i risultati (potenzialmente sovrapposti)
4. Aggrega con RRF o union + deduplicazione per chunk ID
5. Seleziona top-k unici più rilevanti
6. Genera risposta sulla query originale

DIAGRAMMA ARCHITETTURALE



PRO

- + Massimizza la copertura semantica del corpus
- + Facilmente parallelizzabile
- + Robusto su query ambigue o polisemiche

CONTRO

- – N volte il costo di retrieval
- – Deduplicazione necessaria (overhead)
- – Varianti troppo simili sprecare risorse

TOOL E LIBRERIE

LangChain MultiQueryRetriever • LLM per generazione varianti • RRF per fusion

COMPLESSITÀ

★★☆ Media

USE CASE

Ricerca esplorativa, domande polisemiche, assistenti su corpora eterogenei

ESEMPIO PYTHON

```
pip install langchain, openai
```

```python
from langchain.retrievers import MultiQueryRetriever
from langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(temperature=0.3)

# Genera N varianti da prospettive diverse
retriever = MultiQueryRetriever.from_llm(
    retriever=vectordb.as_retriever(
        search_kwargs={"k": 6}),
    llm=llm,
    parser_key="lines"  # splitta su newline
)
# RRF e deduplicazione automatica
docs = retriever.get_relevant_documents(
    "differenze tra RAG e fine-tuning")
print(f"Unique docs: {len(docs)}")
```

RIFERIMENTI BIBLIOGRAFICI

- LangChain MultiQueryRetriever docs. python.langchain.com
- Rackauckas, *RAG Fusion* (2024). arxiv:2402.03367

#12 Query Routing

Query Enhancement Routing / Strategy selection

Key feature: **Classificazione intent → retrieval ottimale**

DESCRIZIONE

Query Routing classifica ogni query per intent o dominio e la instrada alla strategia di retrieval più appropriata. Un router LLM (o un classificatore) decide se usare vector search, BM25, SQL, graph traversal o una API esterna. Questo riduce i costi per query semplici e migliora la qualità per query specializzate, senza esporre la complessità all'utente finale.

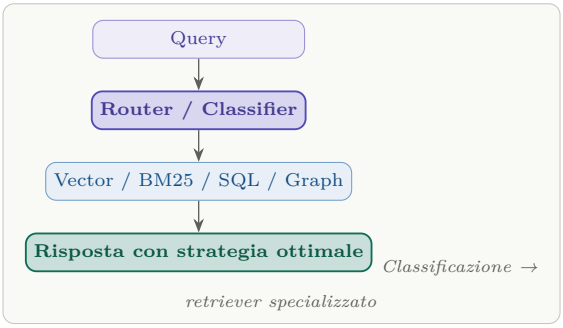
COME FUNZIONA

Un LLM o classificatore leggero analizza la query e ne determina il tipo (lookup, policy, troubleshoot, confronto, calcolo). In base alla classificazione, seleziona il retriever ottimale. I retrievers sono indipendenti e possono essere aggiunti/rimossi senza toccare il core del sistema.

PASSI IMPLEMENTATIVI

1. Classificatore o LLM analizza la query
2. Determina il tipo: lookup / policy / troubleshoot / calcolo / altro
3. Seleziona il retriever ottimale per il tipo
4. Esegui retrieval con la strategia scelta
5. Genera risposta con contesto ottimale

DIAGRAMMA ARCHITETTURALE



PRO

- + Costo ridotto per query semplici
- + Qualità migliore per query specializzate
- + Architettura modulare e scalabile

CONTRO

- – Errori di routing penalizzano la qualità
- – Richiede un catalogo di retriever mantenuto
- – Monitoraggio necessario per rilevare deriva nel routing

TOOL E LIBRERIE

LLM router (GPT-3.5) • LlamaIndex RouterQueryEngine • Intent classifier (BERT-based)

COMPLESSITÀ ★★☆ Media

USE CASE

Assistenti multi-dominio, sistemi con fonti dati eterogenee (SQL + vector + API)

ESEMPIO PYTHON

```
pip install llama-index
```

```python
from llama_index.core.query_engine import RouterQueryEngine
from llama_index.core.selectors import LLMSingleSelector
from llama_index.core.tools import QueryEngineTool

# Definisci retriever specializzati
tools = [
    QueryEngineTool.from_defaults(
        query_engine=vector_engine,
        description="Ricerca semantica su documenti"),
    QueryEngineTool.from_defaults(
        query_engine=sql_engine,
        description="Query su dati strutturati/tabelle"),
]
router = RouterQueryEngine(
    selector=LLMSingleSelector.from_defaults(),
    query_engine_tools=tools,
)
response = router.query(user_query)
```

RIFERIMENTI BIBLIOGRAFICI

- Shen et al., *HuggingGPT* (2023). arxiv:2303.17580
- LlamaIndex Router docs. docs.llamaindex.ai

#13 Contextual Chunk Headers

Context Enrichment Chunk augmentation

Key feature: **Header contestuale document/section prefix**

DESCRIZIONE

Aggiunge a ogni chunk, prima dell'embedding, un header contestuale contenente il titolo del documento e il path della sezione di appartenenza. Questo riduce drasticamente l'ambiguità dei chunk isolati: un chunk che dice 'Il limite è 100mg' è ambiguo da solo, ma con header 'Farmaco X > Dosaggio per adulti' diventa preciso. Tecnica proposta da Anthropic come parte di Contextual Retrieval.

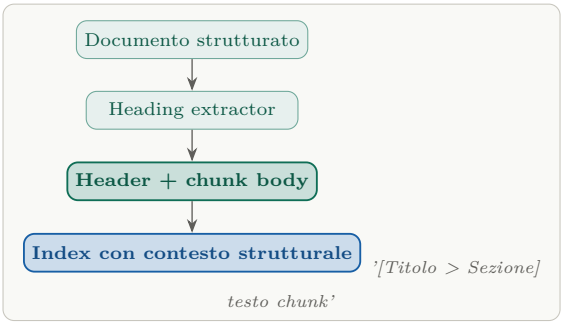
COME FUNZIONA

Durante l'indicizzazione, per ogni chunk si costruisce un header del tipo 'Questo chunk appartiene a [Titolo] (sezione: [Path sezione])'. L'header viene prepeso al testo del chunk prima dell'embedding. Il vettore risultante incorpora sia il contenuto che il contesto strutturale del documento.

PASSI IMPLEMENTATIVI

1. Estrai la struttura del documento (titoli, heading path)
2. Per ogni chunk: costruisci header 'Doc: X, Sezione: Y > Z'
3. Prependi header al testo del chunk
4. Embeds header + testo come unità unica
5. Indicizza nel vector DB
6. A retrieval-time: il chunk recuperato porta con sé il contesto strutturale

DIAGRAMMA ARCHITETTURALE



PRO

- + Migliora la precisione del retrieval
- + Riduce l'ambiguità dei chunk isolati
- + Ottimo per documenti lunghi e strutturati

CONTRO

- – Più token per chunk (overhead 10–20%)
- – Header malformati peggiorano la qualità
- – Richiede struttura heading consistente nel corpus

TOOL E LIBRERIE

Anthropic Contextual Retrieval • Heading extractor (BeautifulSoup, PyPDF2) • Vector DB standard

COMPLESSITÀ ★☆☆ Bassa

USE CASE

Manuali tecnici, policy aziendali, documentazione legale, PDF strutturati

ESEMPIO PYTHON

```
pip install llama-index
```

```python
from llama_index.core.node_parser import (
    SentenceSplitter)

# Estrai heading path da ogni documento
def add_header(doc, nodes):
    title   = doc.metadata.get("title", "Documento")
    section = doc.metadata.get("section", "")
    for node in nodes:
        header = f"[{title} > {section}]\n"
        node.text = header + node.text
    return nodes

parser = SentenceSplitter(chunk_size=512,
                          chunk_overlap=50)
nodes  = parser.get_nodes_from_documents(docs)
nodes  = add_header(doc, nodes)
index  = VectorStoreIndex(nodes)
```

RIFERIMENTI BIBLIOGRAFICI

- Anthropic, *Contextual Retrieval* (2024). anthropic.com

#14 Semantic Chunking

Context Enrichment Chunking (semantic boundaries)

Key feature: **Boundary-aware chunking per topic shift**

DESCRIZIONE

Invece di dividere il testo a dimensione fissa, il Semantic Chunking identifica i confini naturali del testo dove il topic cambia significativamente. Si calcola la similarity tra frasi adiacenti; dove la similarity scende sotto una soglia si inserisce un boundary. Il risultato sono chunk con coerenza semantica interna molto più alta, che migliorano sia la qualità del retrieval che la coerenza della risposta.

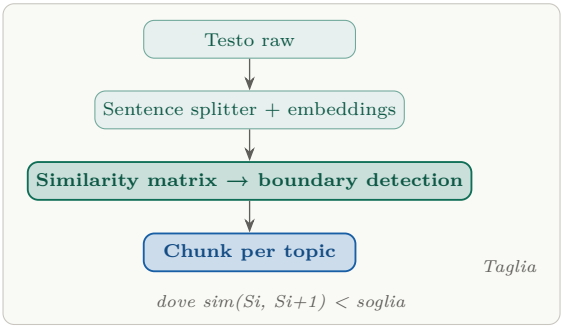
COME FUNZIONA

Si divide il testo in frasi, si calcolano gli embedding di ogni frase, poi si misura la cosine similarity tra frasi adiacenti. Dove la similarity scende sotto una soglia (tipicamente 0.75) si taglia. I gruppi di frasi ad alta similarity vengono aggregati in chunk. La dimensione dei chunk risulta variabile ma semanticamente coerente.

PASSI IMPLEMENTATIVI

1. Dividi il testo in frasi (sentence splitter)
2. Calcola embedding per ogni frase
3. Misura similarity cosine tra frasi adiacenti s_i e s_{i+1}
4. Inserisci boundary dove similarity < 0.75 (soglia tunabile)
5. Aggrega frasi in chunk per topic
6. Indicizza i chunk semantici nel vector DB

DIAGRAMMA ARCHITETTURALE



PRO

- + Chunk più coerenti semanticamente
- + Migliora il retrieval concettuale
- + Riduce la frammentazione delle idee

CONTRO

- – Preprocessing più complesso e lento
- – Possibili errori di boundary su testi complessi
- – Dimensioni chunk variabili (più difficili da gestire)

TOOL E LIBRERIE

sentence-transformers • spaCy per sentence splitting • LlamaIndex SemanticSplitterNodeParser

COMPLESSITÀ ★★☆ Media

USE CASE

Paper scientifici, specifiche tecniche, documenti lunghi con sezioni eterogenee

ESEMPIO PYTHON

```
pip install llama-index, sentence-transformers
```

```python
from llama_index.core.node_parser import (
    SemanticSplitterNodeParser)
from llama_index.embeddings.openai import (
    OpenAIEmbedding)

embed_model = OpenAIEmbedding()

# Chunk basato su similarity tra frasi adiacenti
splitter = SemanticSplitterNodeParser(
    buffer_size=1,            # frasi di contesto
    breakpoint_percentile_threshold=75,  # soglia
    embed_model=embed_model,
)
nodes = splitter.get_nodes_from_documents(docs)
print(f"Chunk: {len(nodes)}, "
      f"avg len: {sum(len(n.text) for n in nodes)//len(nodes)}")
```

RIFERIMENTI BIBLIOGRAFICI

- Kamradt, *Semantic Chunking* (2024). github
- LlamaIndex SemanticSplitter docs. docs.llamaindex.ai

#15 Context Window Enhancement

Context Enrichment Neighbor expansion

Key feature: **Sliding-window neighbor chunk expansion**

DESCRIZIONE

Quando un chunk viene recuperato, spesso il suo significato è incompleto senza i chunk adiacenti (precedente e/o successivo). Context Window Enhancement espande automaticamente il contesto recuperato includendo i chunk vicini nell'ordine originale del documento. Questo ripristina la coerenza locale senza dover modificare la strategia di retrieval.

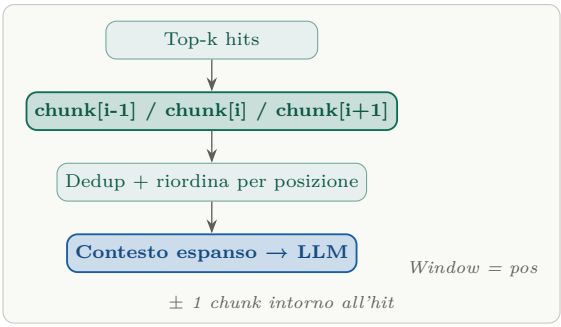
COME FUNZIONA

I chunk vengono indicizzati con metadati di posizione (doc_id, posizione ordinale). Quando un chunk viene recuperato come hit, il sistema recupera automaticamente anche i chunk nelle posizioni $pos-w$ a $pos+w$ (window $w = 1$ di default). I chunk espansi vengono deduplicati e ordinati per posizione prima di essere passati all'LLM.

PASSI IMPLEMENTATIVI

1. Indicizza ogni chunk con metadata: doc_id, pos (indice ordinale)
2. Retrieval standard: ottieni top-k hits
3. Per ogni hit: recupera chunk nelle posizioni $[pos - w, pos + w]$
4. Deduplica i chunk (un hit può generare overlap con un altro)
5. Ordina per doc_id + pos (ordine originale del documento)
6. Usa il contesto espanso per la generazione

DIAGRAMMA ARCHITETTURALE



PRO

- + Ripristina la coerenza locale del testo
- + Semplice da implementare su pipeline esistenti
- + Migliora la comprensione di testi narrativi e tecnici

CONTRO

- – Aumenta il numero di token nel context window
- – Può introdurre rumore da chunk non rilevanti vicini
- – Richiede metadata di posizione a index-time

TOOL E LIBRERIE

Chunk position metadata • Docstore API (LlamaIndex) • LangChain ParentDocumentRetriever

COMPLESSITÀ

★☆☆ Bassa

USE CASE

Documenti tecnici, narrativi, libri, paper scientifici con argomenti interconnessi

ESEMPIO PYTHON

```
pip install llama-index
```

```python
from llama_index.core.node_parser import (
    SentenceWindowNodeParser)
from llama_index.core.postprocessor import (
    MetadataReplacementPostProcessor)

# Indicizza frasi singole, espandi al runtime
parser = SentenceWindowNodeParser.from_defaults(
    window_size=2)          # +/- 2 frasi vicine
nodes  = parser.get_nodes_from_documents(docs)
index  = VectorStoreIndex(nodes)

# Postprocessor sostituisce la frase con la finestra
postproc = MetadataReplacementPostProcessor(
    target_metadata_key="window")
query_engine = index.as_query_engine(
    node_postprocessors=[postproc])
```

RIFERIMENTI BIBLIOGRAFICI

- LlamaIndex SentenceWindowNodeParser. docs.llamaindex.ai

#16 Relevant Segment Extraction

Context Enrichment Context selection / Extraction

Key feature: **Span-level extraction post-retrieval**

DESCRIZIONE

Dopo il retrieval di documenti completi o chunk lunghi, un LLM extractor identifica e restituisce solo gli span di testo minimali che supportano la risposta alla query. Questo massimizza il signal-to-noise ratio nel context window, permettendo di includere evidenze da più documenti nello stesso spazio.

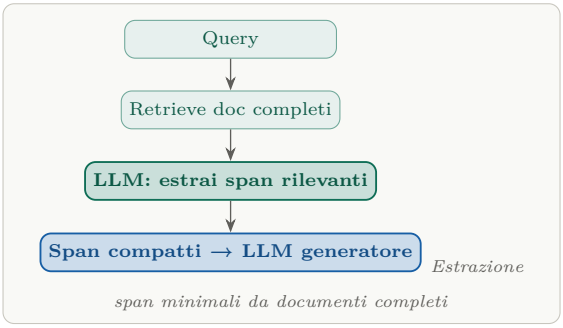
COME FUNZIONA

Si recuperano k documenti completi o chunk lunghi. Per ogni documento, si chiede a un LLM di estrarre 'i passaggi minimi necessari a rispondere a questa query'. I passaggi estratti (span) vengono concatenati e passati all'LLM generatore. La risposta è più precisa e le citazioni sono più accurate.

PASSI IMPLEMENTATIVI

1. Retrieve $k = 8$ documenti completi o chunk lunghi
2. Per ogni documento: LLM extractor — 'estrai i passaggi minimi per: [query]'
3. Ottieni span di testo compatti e pertinenti
4. Concatena gli span (ordinati per rilevanza)
5. Genera risposta con span come contesto
6. Cita gli span estratti con riferimento al documento sorgente

DIAGRAMMA ARCHITETTURALE



PRO

- + Massimo signal-to-noise nel context window
- + Permette più evidenze diverse nello stesso spazio
- + Citazioni più precise e verificabili

CONTRO

- – Latenza aggiunta (LLM call per estrazione)
- – L'extractor può perdere sfumature o contesto necessario
- – Richiede buon parsing per testi con formattazione complessa

TOOL E LIBRERIE

LLM span extractor • LangChain Contextual Compression • Cross-encoder extractors

COMPLESSITÀ ★★☆ Media

USE CASE

Documenti lunghi, ricerca scientifica, budget token stretto, Q&A ad alta precisione

ESEMPIO PYTHON

pip install langchain, openai

```python
from langchain.retrievers import (
    ContextualCompressionRetriever)
from langchain.retrievers.document_compressors import (
    LLMChainExtractor)

# Estrae solo gli span rilevanti per la query
compressor = LLMChainExtractor.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=vectordb.as_retriever(
        search_kwargs={"k": 8})
)
compressed_docs = compression_retriever\
    .get_relevant_documents(user_query)
# Ogni doc e' ora lo span minimale rilevante
```

RIFERIMENTI BIBLIOGRAFICI

- LangChain Contextual Compression Retriever. python.langchain.com

#17 Contextual Compression

Context Enrichment Compression / Summarization

Key feature: **Query-aware compression del contesto recuperato**

DESCRIZIONE

Contextual Compression comprime ogni chunk recuperato in un sommario focalizzato sulla query specifica. A differenza della Relevant Segment Extraction (che estrae span testuali originali), qui si genera un nuovo testo compresso e query-oriented. Questo permette di includere l'essenza di molti più chunk nello stesso context window, aumentando la copertura senza aumentare i token.

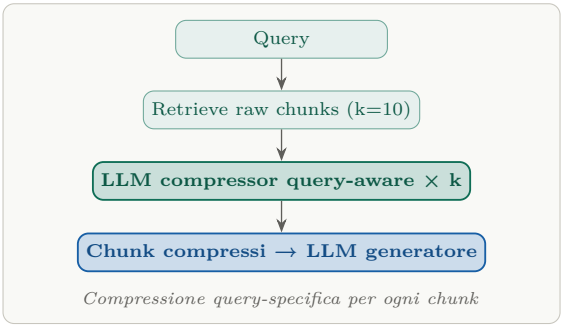
COME FUNZIONA

Ogni chunk recuperato viene passato a un LLM compressor con la query come riferimento. Il compressor genera una versione ridotta del chunk che mantiene solo le informazioni rilevanti per quella specifica query. I chunk compressi sono tipicamente il 20–40% dell'originale. L'insieme dei chunk compressi viene usato come contesto per la generazione.

PASSI IMPLEMENTATIVI

1. Retrieve top-$k = 10$ chunk
2. Per ogni chunk: LLM compressor — 'comprimi mantenendo solo ciò che risponde a: [query]'
3. Ottieni versione compressa query-oriented di ogni chunk
4. Filtra i chunk la cui compressione risulta vuota (irrilevanti)
5. Concatena i chunk compressi nel context window
6. Genera risposta con contesto compresso

DIAGRAMMA ARCHITETTURALE



PRO

+ Più evidenze nel context window
+ Maggiore densità di rilevanza
+ Aiuta modelli con context window piccola

CONTRO

– La compressione può perdere sfumature importanti
– LLM call extra per ogni chunk
– Complessità di attribuzione per le citazioni

TOOL E LIBRERIE

LangChain LLMChainExtractor • LLM compressor (GPT-3.5 per costo) • LlamaIndex SentenceEmbeddingOptimizer

COMPLESSITÀ ★★☆ Media

USE CASE

Corpus con documenti lunghi, modelli con context window limitata, summarization pipeline

ESEMPIO PYTHON

pip install langchain, openai

```python
from langchain.retrievers.document_compressors import (
    LLMChainFilter)
from langchain.retrievers import (
    ContextualCompressionRetriever)

# Filtra chunk irrilevanti (invece di estrarre span)
llm_filter = LLMChainFilter.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
    base_compressor=llm_filter,
    base_retriever=vectordb.as_retriever(
        search_kwargs={"k": 10})
)
docs = compression_retriever\
    .get_relevant_documents(user_query)
print(f"Chunk dopo compressione: {len(docs)}")
```

RIFERIMENTI BIBLIOGRAFICI

- LangChain Contextual Compression docs. python.langchain.com

#18 Document Hierarchy (Parent-Child)

Context Enrichment Hierarchical chunking

Key feature: **Indicizzazione figlio, recupero genitore**

DESCRIZIONE

Il Parent-Child chunking (o Small-to-Big retrieval) affronta il trade-off tra precisione del retrieval (chunk piccoli, più specifici) e coerenza del contesto per la generazione (chunk grandi, più completi). La soluzione: si indicizzano chunk piccoli per il retrieval, ma quando un chunk piccolo viene trovato, si restituisce il suo chunk genitore (più grande) come contesto per la generazione.

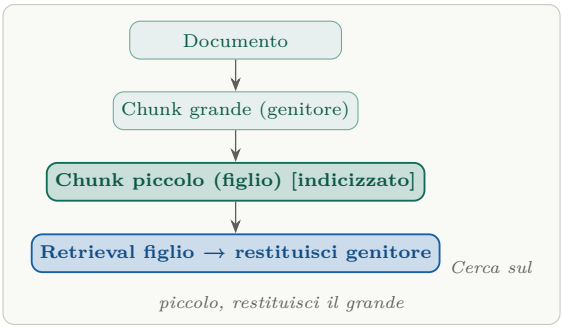
COME FUNZIONA

A index-time, ogni documento viene suddiviso a due livelli: chunk grandi (genitori, 1500 token) e chunk piccoli (figli, 300 token). Solo i chunk figli vengono indicizzati nel vector DB. Ogni chunk figlio ha un riferimento al suo genitore. A runtime, la ricerca avviene sui figli (alta precisione), ma il contesto restituito all'LLM è il genitore (alta coerenza).

PASSI IMPLEMENTATIVI

1. Suddividi il documento in chunk grandi (genitori, 1000–2000 token)
2. Suddividi ogni genitore in chunk piccoli (figli, 200–400 token)
3. Indicizza solo i chunk figli nel vector DB
4. Conserva il mapping figlio→genitore (docstore)
5. A runtime: ricerca sui chunk figli
6. Restituisci il chunk genitore corrispondente come contesto

DIAGRAMMA ARCHITETTURALE



PRO

- + Alta precisione di retrieval (chunk piccoli)
- + Alta coerenza del contesto per la generazione (chunk grandi)
- + Elimina il trade-off classico dimensione chunk

CONTRO

- – Più complesso da indicizzare e gestire
- – Doppia storage (figli + genitori)
- – Mapping figlio-genitore da mantenere consistente

TOOL E LIBRERIE

LangChain ParentDocumentRetriever • LlamaIndex HierarchicalNodeParser • Docstore per chunk genitori

COMPLESSITÀ ★★☆ Media

USE CASE

Documenti tecnici lunghi, manuali, paper dove serve precisione di retrieval + coerenza narrativa

ESEMPIO PYTHON

```
pip install langchain
```

```python
from langchain.retrievers import (
    ParentDocumentRetriever)
from langchain.storage import InMemoryStore
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Chunk figli per retrieval (piccoli)
child_splitter  = RecursiveCharacterTextSplitter(
    chunk_size=300)
# Chunk genitori per contesto (grandi)
parent_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1500)
store     = InMemoryStore()
retriever = ParentDocumentRetriever(
    vectorstore=vectordb, docstore=store,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
retriever.add_documents(docs)
```

RIFERIMENTI BIBLIOGRAFICI

- LangChain ParentDocumentRetriever docs. python.langchain.com
- LlamaIndex Hierarchical docs. docs.llamaindex.ai

#19 Late Chunking

Context Enrichment Chunking post-encoding

Key feature: **Embedding contestuale prima del chunking**

DESCRIZIONE

Late Chunking inverte l'ordine classico: invece di chunkare il documento e poi embedderlo, prima si esegue l'encoding dell'intero documento (o di sezioni lunghe) per ottenere rappresentazioni contestuali di ogni token, e poi si aggregano i token per formare i chunk embedding. Questo permette a ogni chunk di avere embedding che incorporano il contesto dell'intero documento, eliminando il problema degli embedding out-of-context.

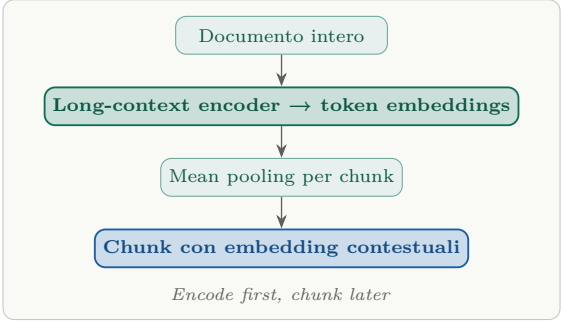
COME FUNZIONA

Si usa un long-context embedding model (es. jina-embeddings-v2) per encodare l'intero documento e ottenere embedding token-by-token contestualizzati sull'intero testo. Poi si applica il chunking sui token embedding tramite mean pooling per ottenere l'embedding di ogni chunk. Il risultato è un chunk embedding che 'conosce' il suo contesto nel documento.

PASSI IMPLEMENTATIVI

1. Usa un long-context embedding model (jina-v2, nomic-embed-text)
2. Encoding dell'intero documento: ottieni token embedding contestuali
3. Applica la strategia di chunking (fixed, semantic) sulle posizioni token
4. Mean pooling degli embedding dei token per ogni chunk
5. Upsert i chunk embedding nel vector DB
6. Retrieval standard: i chunk hanno embedding contestuali

DIAGRAMMA ARCHITETTURALE



PRO

- + Embedding contestuali superiori agli embedding isolati
- + Risolve il problema di chunk ambigui fuori contesto
- + Compatibile con qualsiasi strategia di chunking

CONTRO

- – Richiede long-context embedding model (max 8k–128k token)
- – Più costoso a index-time (encoding intero documento)
- – Non applicabile con modelli di embedding standard a context corto

TOOL E LIBRERIE

jina-embeddings-v2 (8k token) • nomic-embed-text (8k token) • LlamaIndex Late Chunking

COMPLESSITÀ ★★☆ Media

USE CASE

Documenti lunghi con alta interdipendenza tra sezioni, testi legali, contratti, paper

ESEMPIO PYTHON

```
pip install llama-index, torch
```

```python
# Late Chunking con jina-embeddings-v2
from transformers import AutoModel, AutoTokenizer
import torch

model_id = "jinaai/jina-embeddings-v2-base-en"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model     = AutoModel.from_pretrained(model_id)

# 1. Encode intero documento
inputs = tokenizer(doc_text, return_tensors="pt",
                   truncation=True, max_length=8192)
with torch.no_grad():
    token_embs = model(**inputs).last_hidden_state[0]

# 2. Chunk DOPO: mean pooling per ogni chunk
def pool_chunk(start, end):
    return token_embs[start:end].mean(dim=0)
```

RIFERIMENTI BIBLIOGRAFICI

- Günther et al., *Late Chunking: Contextual Chunk Embeddings* (2024). arxiv:2409.04701
- Jina AI, *Late Chunking blog* (2024). jina.ai

#20 Fusion Retrieval

Advanced Retrieval Hybrid / Fusion retrieval

Key feature: **Dense + BM25 fusion con Reciprocal Rank Fusion**

DESCRIZIONE

Fusion Retrieval combina i punti di forza del retrieval denso (semantico, robusto su parafrasature) e del retrieval sparso BM25 (preciso su termini esatti, jargon tecnico, codici errore). I risultati vengono fusi tramite Reciprocal Rank Fusion (RRF), un algoritmo che combina ranking eterogenei senza richiedere la calibrazione dei punteggi assoluti. Il risultato è un retrieval robusto su tutti i tipi di query.

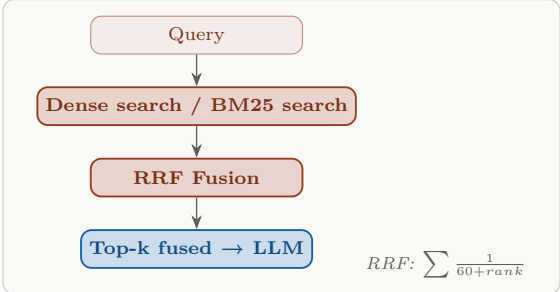
COME FUNZIONA

Si mantengono due indici in parallelo: un vector DB per la ricerca densa e un motore BM25 (Elasticsearch, Solr) per la ricerca sparsa. Ad ogni query, si eseguono entrambe le ricerche e si ottengono due ranked list. RRF combina le due liste assegnando a ogni documento uno score $\sum_r \frac{1}{k+r}$ dove r è il rank nella lista e $k = 60$ è un parametro.

PASSI IMPLEMENTATIVI

1. Mantieni vector DB (dense) e BM25 index (sparse) in parallelo
2. Query: esegui dense search (top-20) e BM25 search (top-20) in parallelo
3. Calcola RRF score: $\sum_{L \in \{dense, sparse\}} \frac{1}{60 + rank_L(doc)}$
4. Ordina tutti i documenti per RRF score decrescente
5. Prendi i top-k (tipicamente 6–10)
6. Genera risposta con i risultati fusi

DIAGRAMMA ARCHITETTURALE



PRO

- + Recall robusto su tutti i tipi di query
- + Gestisce sia match semantici che esatti
- + Non richiede calibrazione dei punteggi assoluti

CONTRO

- – Richiede due indici da mantenere
- – Latenza leggermente maggiore (due ricerche parallele)
- – Configurazione infrastrutturale più complessa

TOOL E LIBRERIE

Elasticsearch / OpenSearch (BM25) • Vector DB (Qdrant, Weaviate, Chroma) • Pinecone Hybrid Search

COMPLESSITÀ ★★☆ Media

USE CASE

Support (codici errore), log analysis, documentazione tecnica con gergo specifico

ESEMPIO PYTHON

```
pip install langchain, rank-bm25
```

```python
from langchain.retrievers import BM25Retriever, EnsembleRetriever

# BM25 (sparse) + Vector (dense)
bm25_retriever = BM25Retriever.from_documents(docs)
bm25_retriever.k = 20

dense_retriever = vectordb.as_retriever(
    search_kwargs={"k": 20})

# RRF fusion automatica con pesi configurabili
ensemble = EnsembleRetriever(
    retrievers=[bm25_retriever, dense_retriever],
    weights=[0.4, 0.6]  # BM25 40%, dense 60%
)
docs = ensemble.get_relevant_documents(query)
```

RIFERIMENTI BIBLIOGRAFICI

- Cormack et al., *RRF* (SIGIR 2009). dl.acm.org
- Pinecone, *Hybrid Search* (2024). pinecone.io

#21 Reranking

Advanced Retrieval Ranking / Relevance modeling

Key feature: **Second-stage cross-encoder reranking**

DESCRIZIONE

Il Reranking separa il retrieval in due fasi: una fase di recall (bi-encoder, veloce, recupera molti candidati) e una fase di precision (cross-encoder, lento ma preciso, riordina i candidati). I cross-encoder valutano la coppia (query, documento) congiuntamente, catturando interazioni fine-grained impossibili con i bi-encoder. Il guadagno di precisione è tipicamente significativo (+10–20% MRR).

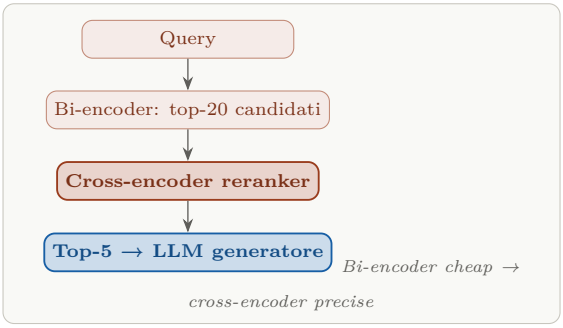
COME FUNZIONA

Si esegue prima una ricerca vettoriale con un bi-encoder per ottenere 20–50 candidati (alta recall). Poi ogni coppia (query, chunk) viene valutata da un cross-encoder che assegna uno score di rilevanza. I candidati vengono riordinati per score e solo i top-5 vengono passati all'LLM. Il cross-encoder è computazionalmente costoso ma operato su un set limitato di candidati.

PASSI IMPLEMENTATIVI

1. Vector search bi-encoder: ottieni top-20/50 candidati
2. Per ogni coppia (query, chunk): score cross-encoder
3. Ordina i candidati per score decrescente
4. Prendi top-5 come contesto per la generazione
5. Genera risposta con il contesto di alta qualità

DIAGRAMMA ARCHITETTURALE



PRO

- + Guadagno di precisione significativo (+10–20% MRR)
- + Meno contesto irrilevante passato all'LLM
- + Risposte più grounded e precise

CONTRO

- – Cross-encoder computazionalmente costoso
- – Latenza aggiuntiva (proporzionale ai candidati)
- – Richiede infrastruttura per il modello reranker

TOOL E LIBRERIE

bge-reranker-v2-m3 • Cohere Rerank API • cross-encoder/ms-marco-MiniLM (HuggingFace)

COMPLESSITÀ ★★☆ Media

USE CASE

Q&A ad alta precisione, sistemi legali/medici, search enterprise

ESEMPIO PYTHON

pip install sentence-transformers, cohere

```python
from sentence_transformers import CrossEncoder

# Bi-encoder: retrieval veloce (top-20)
candidates = vectordb.similarity_search(query, k=20)

# Cross-encoder: reranking preciso
reranker = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L-6-v2")
pairs  = [(query, doc.page_content)
          for doc in candidates]
scores = reranker.predict(pairs)

# Riordina per score decrescente, prendi top-5
ranked = sorted(zip(scores, candidates),
                key=lambda x: x[0], reverse=True)
top5   = [doc for _, doc in ranked[:5]]
```

RIFERIMENTI BIBLIOGRAFICI

- Nogueira & Cho, *Passage Re-ranking with BERT* (2019). arxiv:1901.04085
- Cohere Rerank docs. docs.cohere.com

#22 Multi-faceted Filtering

Advanced Retrieval Metadata & rule-based filtering

Key feature: **Filtri ACL/tenant/tempo pre e post retrieval**

DESCRIZIONE

In sistemi enterprise multi-tenant, il retrieval deve rispettare vincoli di accesso, personalizzazione e rilevanza contestuale. Multi-faceted Filtering applica filtri sui metadati dei documenti (tenant, ruolo utente, prodotto, data, lingua) prima o dopo la ricerca vettoriale. Garantisce compliance, personalizzazione e sicurezza senza modificare il corpus o i modelli.

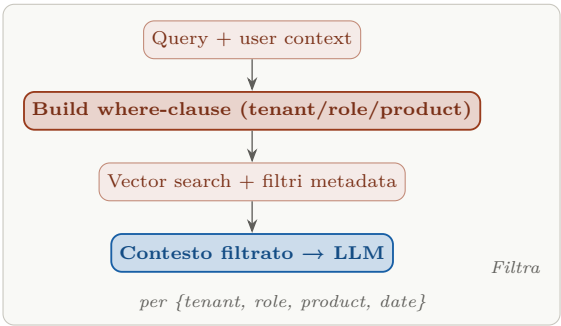
COME FUNZIONA

I documenti vengono indicizzati con metadati strutturati ricchi (tenant, dipartimento, ruolo autorizzato, data, prodotto, lingua). A runtime, il sistema estrae il contesto utente (dall'autenticazione) e costruisce una where-clause per la ricerca vettoriale. I filtri possono essere applicati pre-retrieval (più efficiente) o post-retrieval (più flessibile).

PASSI IMPLEMENTATIVI

1. Indicizza documenti con metadata ricchi: tenant, role, product, date, lang
2. A runtime: estrai user context (JWT, session)
3. Costruisci where-clause: {tenant: X, role: Y, product: Z}
4. Esegui vector search con filtri applicati
5. Opzionale: post-filtering su risultati per regole aggiuntive
6. Genera risposta su contesto filtrato e sicuro

DIAGRAMMA ARCHITETTURALE



PRO

- + Compliance e access control garantiti
- + Personalizzazione automatica per utente
- + Migliora la rilevanza contestuale

CONTRO

- – Over-filtering può ridurre il recall
- – Metadata devono essere completi e accurati
- – Complessità a ingestion time

TOOL E LIBRERIE

Vector DB con metadata filters (Qdrant, Weaviate, Chroma)

- RBAC / ABAC
- JWT per user context extraction

COMPLESSITÀ ★★☆ Media

USE CASE

Enterprise multi-tenant, sistemi con dati sensibili, assistenti personalizzati per ruolo

ESEMPIO PYTHON

pip install qdrant-client, langchain

```python
from qdrant_client import QdrantClient, models

client = QdrantClient(url="http://localhost:6333")

# Filtro multi-attributo prima della ricerca vettoriale
results = client.search(
    collection_name="kb",
    query_vector=query_embedding,
    query_filter=models.Filter(
        must=[
            models.FieldCondition(
                key="tenant",
                match=models.MatchValue(value="acme")),
            models.FieldCondition(
                key="role",
                match=models.MatchAny(
                    any=["admin", "editor"])),
        ]
    ),
    limit=8,
)
```

RIFERIMENTI BIBLIOGRAFICI

- Weaviate, *Filtered Vector Search* (2024). weaviate.io
- Qdrant filtering docs. qdrant.tech

#23 Hierarchical Indices

Advanced Retrieval Hierarchical retrieval coarse→fine

Key feature: **Doc → section → chunk retrieval progressivo**

DESCRIZIONE

Con corpus di grandi dimensioni (migliaia di documenti), un retrieval flat su tutti i chunk diventa rumoroso e costoso. Hierarchical Indices struttura il corpus su tre livelli: documenti (summary), sezioni, chunk. Il retrieval procede coarse-to-fine: prima trova i documenti più rilevanti, poi le sezioni in quei documenti, infine i chunk nelle sezioni pertinenti. Questo riduce lo spazio di ricerca a ogni step.

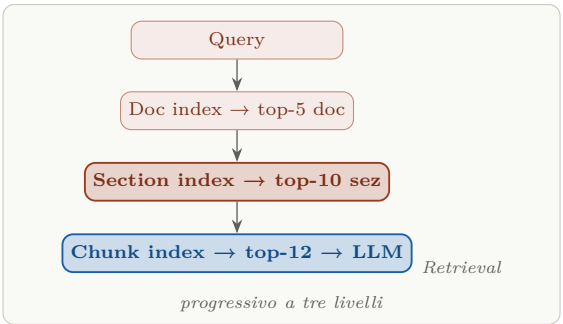
COME FUNZIONA

Si costruiscono tre indici separati: document index (embedding dei summary dei documenti), section index (embedding delle sezioni, filtrato per doc_id), chunk index (embedding dei chunk, filtrato per section_id). Il retrieval usa i risultati di ogni livello come filtro per il livello successivo.

PASSI IMPLEMENTATIVI

1. Costruisci document index: summary di ogni documento
2. Costruisci section index: embedding di ogni sezione, con doc_id
3. Costruisci chunk index: embedding di ogni chunk, con section_id
4. Retrieval L1: top-5 documenti rilevanti
5. Retrieval L2: top-10 sezioni nei documenti trovati
6. Retrieval L3: top-12 chunk nelle sezioni trovate → LLM

DIAGRAMMA ARCHITETTURALE



PRO

- + Scala su corpus di grandi dimensioni
- + Migliore coerenza (retrieval contestualizzato)
- + Riduce lo spazio di ricerca a ogni livello

CONTRO

- – Tre indici da mantenere e aggiornare
- – Più step di latenza (tre retrieval sequenziali)
- – Mapping gerarchia da costruire a index-time

TOOL E LIBRERIE

LlamaIndex HierarchicalNodeParser • Custom multi-index architecture • Docstore per il mapping gerarchia

COMPLESSITÀ ★★★ Alta

USE CASE

Knowledge base aziendali grandi, corpus di documenti tecnici con struttura gerarchica

ESEMPIO PYTHON

```
pip install llama-index
```

```python
from llama_index.core import (
    VectorStoreIndex, SummaryIndex)
from llama_index.core.retrievers import (
    RouterRetriever)

# Indice L1: summary di ogni documento
summary_index = SummaryIndex(doc_nodes)
# Indice L2: chunk dei documenti
vector_index  = VectorStoreIndex(chunk_nodes)

# Retrieval gerarchico: prima i doc rilevanti,
# poi i chunk in quei documenti
summary_retriever = summary_index.as_retriever(
    similarity_top_k=3)
chunk_retriever   = vector_index.as_retriever(
    similarity_top_k=8)
# Filtra chunk per doc_id dei summary trovati
```

RIFERIMENTI BIBLIOGRAFICI

- LlamaIndex Hierarchical docs. docs.llamaindex.ai
- Gao et al., *RAG Survey* (2024). arxiv:2312.10997

#24 Ensemble Retrieval

Advanced Retrieval Retriever ensembling

Key feature: **Dense + sparse + domain-specific ensemble**

DESCRIZIONE

Ensemble Retrieval combina più retrievers eterogenei (dense, sparse, domain-specific, temporale) in un sistema unico. A differenza di Fusion Retrieval (che combina solo dense + BM25), Ensemble può includere retrievers specializzati: un retriever domain-specific per terminologia tecnica, un retriever temporale per informazioni recenti, un retriever semantico generale. La fusione avviene tramite RRF o pesi configurabili.

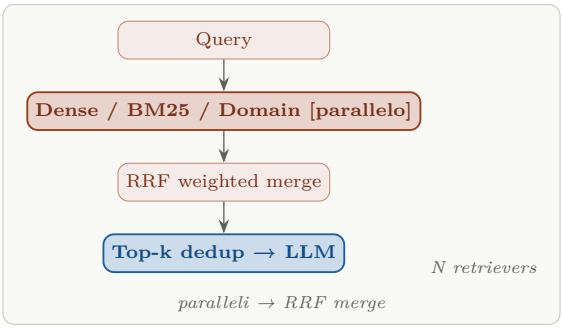
COME FUNZIONA

Ogni retriever opera indipendentemente sulla stessa query, producendo una ranked list. Le liste vengono aggregate tramite RRF o weighted merge (con pesi configurati in base alla confidenza di ogni retriever). Il risultato finale viene deduplicato. I retrievers possono essere aggiunti o rimossi senza modificare la pipeline.

PASSI IMPLEMENTATIVI

1. Definisci il pool di retrievers: dense, BM25, domain, temporal
2. Esegui ogni retriever in parallelo sulla query
3. Raccogli tutte le ranked lists
4. Aggrega con RRF o weighted merge
5. Deduplica per chunk ID
6. Seleziona top-k unici come contesto

DIAGRAMMA ARCHITETTURALE



PRO

- + Robustezza su tutti gli stili di query
- + Recall migliorato (blind spots ridotti)
- + Modulare: retrievers aggiungibili senza impatto

CONTRO

- – Latenza: N retrieval in parallelo
- – Deduplicazione e merge aggiungono overhead
- – Tuning dei pesi per il weighted merge

TOOL E LIBRERIE

LangChain EnsembleRetriever • Multiple embedding models • BM25 + dense + domain-specific

COMPLESSITÀ ★★★ Alta

USE CASE

Corpora eterogenei con dati strutturati e non strutturati, domini specializzati

ESEMPIO PYTHON

```
pip install langchain, rank-bm25
```

```python
from langchain.retrievers import EnsembleRetriever, BM25Retriever
from langchain.vectorstores import Chroma

# Pool di retrievers eterogenei
dense    = Chroma(...).as_retriever(search_kwargs={"k":15})
sparse   = BM25Retriever.from_documents(docs, k=15)
domain   = domain_vectordb.as_retriever(
               search_kwargs={"k": 10})

# Ensemble con pesi per retriever
ensemble = EnsembleRetriever(
    retrievers=[dense, sparse, domain],
    weights=[0.5, 0.3, 0.2]
)
docs = ensemble.get_relevant_documents(query)
# RRF automatico, dedup incluso
```

RIFERIMENTI BIBLIOGRAFICI

- LangChain EnsembleRetriever docs. python.langchain.com

#25 Dartboard Retrieval (MMR)

Advanced Retrieval Diversified retrieval / Coverage-first

Key feature: **Maximum Marginal Relevance per copertura**

DESCRIZIONE

Maximum Marginal Relevance (MMR) bilancia rilevanza e diversità nella selezione dei chunk da passare all'LLM. Il retrieval standard tende a recuperare chunk molto simili tra loro (ridondanti). MMR seleziona iteratively il prossimo chunk che massimizza $\lambda \cdot sim(q, d_i) - (1-\lambda) \cdot \max_{d_j \in S} sim(d_i, d_j)$: rilevante rispetto alla query ma diverso dai chunk già selezionati.

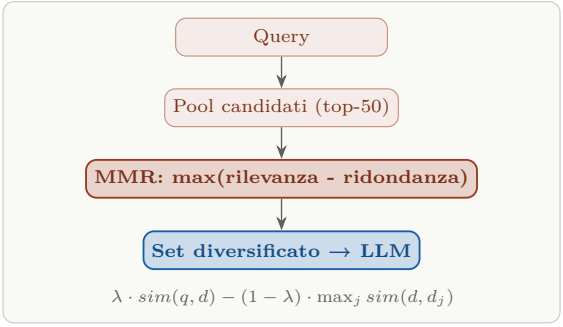
COME FUNZIONA

Si parte da un pool ampio di candidati (top-50 dalla ricerca vettoriale). MMR seleziona iteratively: il primo chunk è il più rilevante per la query; ogni chunk successivo massimizza il trade-off tra rilevanza e diversità rispetto ai chunk già selezionati. Il parametro λ (tipicamente 0.5) bilancia i due obiettivi.

PASSI IMPLEMENTATIVI

1. Retrieve pool ampio di candidati (top-50)
2. Primo chunk: argmax similarità con la query
3. Per ogni chunk successivo: $\text{argmax}_{d \notin S} [\lambda \cdot sim(q, d) - (1 - \lambda) \cdot \max_j sim(d, d_j)]$
4. Ripeti fino a k chunk selezionati
5. Usa il set diversificato come contesto
6. Tuna λ: 1.0 = pura rilevanza, 0.0 = pura diversità

DIAGRAMMA ARCHITETTURALE



PRO

- + Migliore copertura topica per query broad
- + Riduce la ridondanza nel contesto
- + Migliore per domande esplorative

CONTRO

- – Può ridurre la precisione per query narrow
- – Computazione iterativa (più lenta di top-k)
- – Il parametro λ richiede tuning

TOOL E LIBRERIE

LangChain MMR retrieval • Qdrant MMR search • FAISS + custom MMR implementation

COMPLESSITÀ ★★☆ Media

USE CASE

Ricerca esplorativa, query broad, riassunti di argomento, discovery su corpus

ESEMPIO PYTHON

```
pip install langchain, faiss-cpu
```

```python
from langchain.vectorstores import FAISS

vectordb  = FAISS.from_documents(docs, embeddings)

# MMR: lambda_mult=0.5 bilancia rilevanza/diversita'
# fetch_k: pool iniziale di candidati
retriever = vectordb.as_retriever(
    search_type="mmr",
    search_kwargs={
        "k": 6,            # chunk da restituire
        "fetch_k": 50,     # pool iniziale
        "lambda_mult": 0.5  # 0=diversity, 1=relevance
    }
)
docs = retriever.get_relevant_documents(
    "tecniche di chunking per RAG")
```

RIFERIMENTI BIBLIOGRAFICI

- Carbonell & Goldstein, *MMR* (SIGIR 1998). dl.acm.org
- LangChain MMR docs. python.langchain.com

#26 Multi-modal RAG

Advanced Retrieval Multimodal RAG

Key feature: **Vision captioning + cross-modal retrieval**

DESCRIZIONE

Il Multi-modal RAG estende il RAG a fonti non testuali: immagini, diagrammi, tabelle, screenshot, grafici. Le immagini vengono processate da vision models che generano caption testuali o embedding multimodali (CLIP). Le caption vengono indicizzate nel vector DB insieme al testo, permettendo retrieval cross-modale: una query testuale può recuperare immagini rilevanti e viceversa.

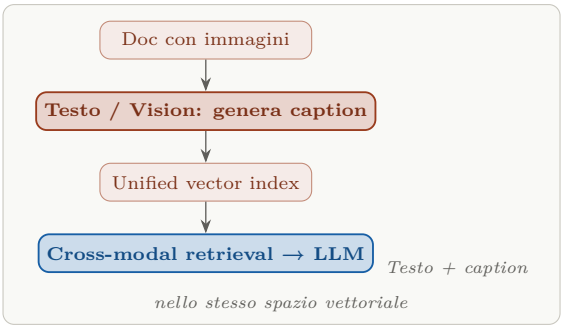
COME FUNZIONA

A index-time: le immagini vengono estratte dai documenti (PDF, slide) e passate a un vision model che genera una caption dettagliata. La caption viene indicizzata con metadata che puntano all'immagine originale. A retrieval-time: la ricerca avviene sulle caption. La risposta finale può includere riferimenti alle immagini originali.

PASSI IMPLEMENTATIVI

1. Estrai immagini/diagrammi dai documenti (PyMuPDF, python-pptx)
2. Genera caption con vision model (GPT-4o, BLIP-2, LLaVA)
3. Indicizza caption con metadata image_id e sorgente
4. Opzionale: usa CLIP per embedding multimodali diretti
5. Retrieval: cerca caption rilevanti per la query
6. Genera risposta con testo + riferimenti immagine

DIAGRAMMA ARCHITETTURALE



PRO

- + Sblocca la conoscenza visiva in PDF e slide
- + Migliora significativamente Q&A su manuali tecnici
- + Gestisce diagrammi, grafici, tabelle complesse

CONTRO

- – Qualità della caption limita la qualità del retrieval
- – Costo computazionale e storage per vision models
- – Richiede vision model di qualità per caption accurate

TOOL E LIBRERIE

GPT-4o / LLaVA per captioning • CLIP embeddings • PyMuPDF per estrazione immagini

COMPLESSITÀ ★★★ Alta

USE CASE

Manuali tecnici, presentazioni, documentazione con diagrammi, screenshot di software

ESEMPIO PYTHON

pip install openai, pymupdf

```python
import fitz  # PyMuPDF
import base64
from openai import OpenAI

client = OpenAI()

def caption_image(img_bytes: bytes) -> str:
    b64 = base64.b64encode(img_bytes).decode()
    resp = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role":"user","content":[
            {"type":"image_url","image_url":
                {"url":f"{b64}"}},
            {"type":"text","text":
                "Descrivi questo diagramma in dettaglio."}
        ]}])
    return resp.choices[0].message.content

# Estrai immagini da PDF e genera caption
doc = fitz.open("manual.pdf")
for page in doc:
    for img in page.get_images():
        caption = caption_image(img_bytes)
```

RIFERIMENTI BIBLIOGRAFICI

- Li et al., *BLIP-2* (2023). arxiv:2301.12597
- OpenAI, *GPT-4V System Card* (2023). openai.com

#27 ColBERT / Token-level Retrieval

Advanced Retrieval Token-level late interaction

Key feature: **Late interaction per rilevanza fine-grained**

DESCRIZIONE

ColBERT (Contextualized Late Interaction over BERT) è un'architettura di retrieval che mantiene embedding a livello di token (non di intero documento) e calcola la rilevanza come MaxSim: per ogni token della query, trova il token del documento con la massima similarità e somma questi score. Questo permette interazioni fine-grained query-documento senza il costo computazionale dei cross-encoder.

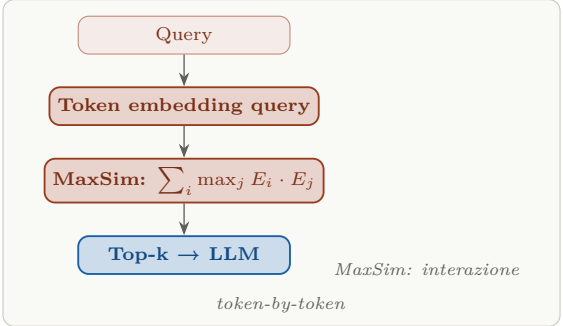
COME FUNZIONA

A index-time, ogni documento viene encodato a livello di token e tutti i token embedding vengono conservati. A runtime, la query viene encodata a livello di token. Il punteggio ColBERT di una coppia (query, doc) è $\sum_{i \in q} \max_{j \in d} E_i \cdot E_j$ (MaxSim). ColBERT può essere usato come retriever o reranker.

PASSI IMPLEMENTATIVI

1. Indica documenti a livello di token con ColBERT encoder
2. Conserva tutti i token embedding per documento
3. A runtime: encoda la query a livello di token
4. Calcola MaxSim: per ogni token query, max similarity con token documento
5. Score finale: somma dei MaxSim per tutti i token query
6. Top-k documenti per MaxSim score → LLM

DIAGRAMMA ARCHITETTURALE



PRO

- \+ Interazioni fine-grained senza costo cross-encoder
- \+ Più preciso di bi-encoder, più veloce di cross-encoder
- \+ Ottimo come primo stage o reranker leggero

CONTRO

- – Storage molto elevato (token embedding per ogni documento)
- – Richiede infrastruttura specializzata (RAGatouille, Vespa)
- – Modello ColBERT più grande da servire

TOOL E LIBRERIE

ColBERT v2 (Stanford NLP) • RAGatouille (Python library) • Vespa, Weaviate (ColBERT support)

COMPLESSITÀ ★★★ Alta

USE CASE

Retrieval ad alta precisione, legal search, medical literature, sistemi dove la precisione supera il costo

ESEMPIO PYTHON

```
pip install ragatouille
```

```python
from ragatouille import RAGPretrainedModel

# ColBERT: indicizzazione token-level
RAG = RAGPretrainedModel.from_pretrained(
    "colbert-ir/colbertv2.0")

# Index (una tantum, poi si riusa)
RAG.index(
    collection=corpus_texts,
    index_name="rag_catalog",
    max_document_length=512,
    split_documents=True,
)
# Retrieval con MaxSim token-level
results = RAG.search(
    query="late interaction retrieval",
    k=5,
    index_name="rag_catalog"
)
```

RIFERIMENTI BIBLIOGRAFICI

- Khattab & Zaharia, *ColBERT* (SIGIR 2020). arxiv:2004.12832
- Santhanam et al., *ColBERTv2* (2021). arxiv:2112.01488
- RAGatouille docs. github.com

#28 Iterative Retrieval

Iterative Multi-step retrieval

Key feature: **Loop retrieve → analyze → retrieve**

DESCRIZIONE

Iterative Retrieval esegue multiple round di recupero, dove ogni round è guidato da un'analisi del contesto già recuperato. Dopo ogni retrieval, un LLM valuta se l'evidenza è sufficiente o propone una follow-up query per colmare le lacune. Il processo si ripete fino a convergenza o al raggiungimento del numero massimo di round. Ideale per domande che richiedono la triangolazione di informazioni da più fonti.

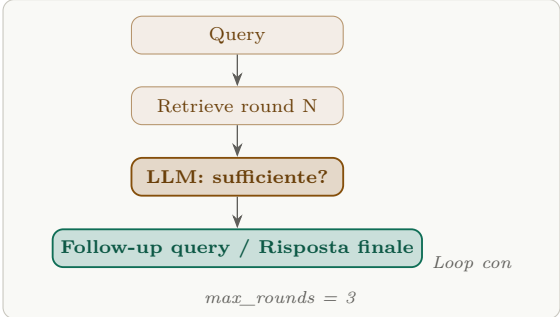
COME FUNZIONA

Il primo retrieval usa la query originale. Dopo ogni round, un LLM analizza il contesto accumulato e decide: l'evidenza è sufficiente per rispondere? Se no, genera una follow-up query che copre l'aspetto mancante. Il contesto si accumula ad ogni round. Al termine, l'LLM genera la risposta finale sull'insieme di tutti i contesti recuperati deduplicati.

PASSI IMPLEMENTATIVI

1. Round 1: retrieve con la query originale
2. LLM: analizza il contesto — è sufficiente? Se sì: vai alla risposta
3. Se no: LLM genera follow-up query specifica
4. Round 2: retrieve con la follow-up query
5. Accumula contesto (deduplica)
6. Ripeti fino a max round (tipicamente 3) o evidenza sufficiente
7. Genera risposta finale su tutto il contesto accumulato

DIAGRAMMA ARCHITETTURALE



PRO

- + Forte su domande complesse e multi-hop
- + Colma autonomamente le lacune di evidenza
- + Robusto rispetto al one-shot retrieval

CONTRO

- – Latenza elevata (N round × retrieval + LLM)
- – Rischio di loop su query circolari
- – Costo proporzionale ai round

TOOL E LIBRERIE

LLM per follow-up generation • Retriever (any) • Caching per evitare retrieval duplicati

COMPLESSITÀ ★★★ Alta

USE CASE

Ricerca scientifica, root cause analysis, domande multi-hop, Q&A su corpora distribuiti

ESEMPIO PYTHON

pip install langchain, openai

```python
MAX_ROUNDS = 3
context, round_n = [], 0

while round_n < MAX_ROUNDS:
    new_docs = retriever.get_relevant_documents(query)
    context += [d for d in new_docs
                if d not in context]  # dedup
    # LLM valuta se le evidenze sono sufficienti
    check = llm.invoke(
        f"Le seguenti evidenze bastano per rispondere "
        f"a '{query}'? Se no, suggerisci follow-up.\n"
        + "\n".join(d.page_content for d in context))
    if "SUFFICIENTE" in check.upper(): break
    query     = check   # usa il follow-up come nuova query
    round_n  += 1

answer = qa_chain.run({"question": orig_query,
                       "context": context})
```

RIFERIMENTI BIBLIOGRAFICI

- Trivedi et al., *IRCoT* (2023). arxiv:2212.10509
- Shao et al., *ITER-RETGEN* (2023). arxiv:2305.15294

#29 Adaptive Retrieval

Iterative Routing / Strategy selection

Key feature: **Classificazione intent → retriever ottimale**

DESCRIZIONE

Adaptive Retrieval instrada ogni query alla strategia di retrieval più appropriata in base all'intent rilevato. Un classificatore (LLM leggero o modello BERT) analizza la query e la categorizza (policy, troubleshoot, definition, calculation, comparison). Ogni categoria ha una strategia ottimale: ricerca ACL per policy, ibrida per troubleshoot, semplice per definizioni. Questo riduce i costi per query semplici e massimizza la qualità per query complesse.

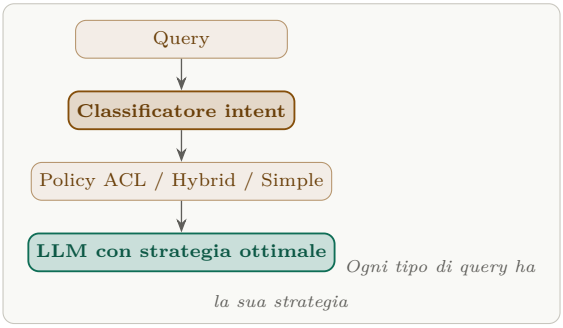
COME FUNZIONA

Un classificatore leggero analizza la query e ne determina il tipo. Un router mappa il tipo alla strategia ottimale. Le strategie sono configurate separatamente e possono essere aggiornate indipendentemente. Il routing può essere basato su regole (keyword matching), su modello (fine-tuned classifier) o su LLM (zero-shot).

PASSI IMPLEMENTATIVI

1. Classificatore (LLM o BERT): analizza la query
2. Determina il tipo: policy/troubleshoot/definition/comparison/other
3. Router: mappa tipo → strategia retrieval
4. Esegui la strategia scelta
5. Genera risposta con contesto ottimale per il tipo

DIAGRAMMA ARCHITETTURALE



PRO

- + Costo ridotto per query semplici
- + Qualità migliore per query specializzate
- + Architettura modulare e estendibile

CONTRO

- – Errori di classificazione penalizzano la qualità
- – Catalogo di strategie da progettare e mantenere
- – Monitoraggio necessario per rilevare deriva

TOOL E LIBRERIE

Intent classifier (BERT, LLM) • LlamaIndex RouterQueryEngine • LangChain routing chains

COMPLESSITÀ

★★☆ Media

USE CASE

Assistenti multi-dominio, sistemi enterprise con workload eterogeneo

ESEMPIO PYTHON

```
pip install llama-index
```

```python
from llama_index.core.query_engine import RouterQueryEngine
from llama_index.core.selectors import LLMSingleSelector

# Retriever per ogni tipo di intent
tools = [
    QueryEngineTool.from_defaults(
        vector_engine, description="policy e normative"),
    QueryEngineTool.from_defaults(
        hybrid_engine, description="troubleshooting"),
    QueryEngineTool.from_defaults(
        sql_engine,    description="dati e statistiche"),
]
adaptive_router = RouterQueryEngine(
    selector=LLMSingleSelector.from_defaults(),
    query_engine_tools=tools,
    verbose=True,   # logga il routing scelto
)
response = adaptive_router.query(user_query)
```

RIFERIMENTI BIBLIOGRAFICI

- LlamaIndex RouterQueryEngine. docs.llamaindex.ai
- Shen et al., *HuggingGPT* (2023). arxiv:2303.17580

#30 Retrieval with Feedback Loop

Iterative Online learning / Feedback

Key feature: **Feedback utente per reranker tuning continuo**

DESCRIZIONE

Questo sistema usa il feedback implicito ed esplicito degli utenti per migliorare continuamente il ranking e il retrieval. Ogni interazione (click, thumbs up/down, dwell time) viene loggata come segnale di feedback. Periodicamente, il reranker viene fine-tuned su questi segnali, allineando il sistema all'intent reale degli utenti. È il meccanismo che trasforma l'uso in produzione in dati di training.

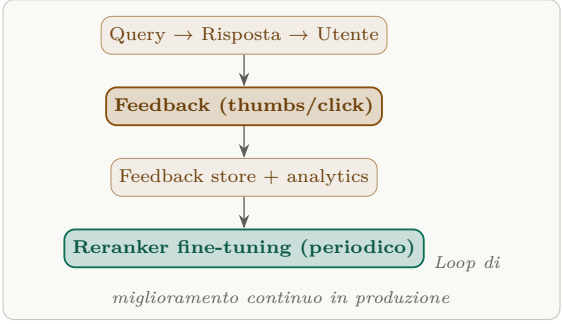
COME FUNZIONA

Ogni query-risposta-feedback viene memorizzata in un feedback store. Il feedback (positivo o negativo) viene associato ai chunk usati come contesto. Periodicamente (daily/weekly) si fine-tuna il reranker usando i feedback come segnali di rilevanza. Si monitora il drift del modello per evitare overfitting su pattern recenti.

PASSI IMPLEMENTATIVI

1. Logga ogni interazione: {query, context_chunks, risposta, user_id}
2. Raccoglie feedback: thumbs up/down, click, dwell time
3. Mappa il feedback ai chunk usati come segnale di rilevanza
4. Periodicamente: costruisci training set da feedback
5. Fine-tuna il reranker (cross-encoder) su training set
6. Deploy del reranker aggiornato; monitora le metriche

DIAGRAMMA ARCHITETTURALE



PRO

+ Il sistema migliora continuamente con l'uso
+ Si allinea all'intent reale degli utenti
+ Identifica automaticamente i gap nella knowledge base

CONTRO

– Richiede UX per raccogliere feedback
– Label rumorose (utenti incoerenti)
– Governance e privacy da gestire sul feedback

TOOL E LIBRERIE

Feedback store (DB) • Reranker fine-tuning (HuggingFace Trainer) • LangSmith / Langfuse per tracing

COMPLESSITÀ ★★★ Alta

USE CASE

Support assistant in produzione, sistemi con alto volume di query, chatbot aziendali

ESEMPIO PYTHON

```
pip install langfuse, sentence-transformers
```

```python
from langfuse import Langfuse

lf = Langfuse()

# Logga ogni interazione con il feedback utente
def log_feedback(query, context, answer, score):
    trace = lf.trace(name="rag-pipeline")
    trace.generation(input=query, output=answer,
                     metadata={"context": context})
    trace.score(name="user-feedback", value=score)

# Fine-tuning periodico del reranker
# (weekly cron job)
from sentence_transformers import (
    CrossEncoder, InputExample, losses)
training_data = build_training_from_feedback(lf)
model = CrossEncoder("cross-encoder/ms-marco-base")
model.fit(training_data, epochs=1)
```

RIFERIMENTI BIBLIOGRAFICI

- Ouyang et al., *RLHF / InstructGPT* (2022). arxiv:2203.02155
- Langfuse docs. langfuse.com

#31 FLARE

Iterative Active retrieval generation

Key feature: **Forward-looking active retrieval**

DESCRIZIONE

FLARE (Forward-Looking Active REtrieval) è una tecnica iterativa dove l'LLM genera la risposta token per token e, quando incontra un passaggio a bassa confidenza (token probability bassa), si ferma e esegue un retrieval attivo per ottenere le informazioni mancanti. Il retrieval è 'forward-looking': la query usata per il retrieval è costruita dai token generati finora, che rappresentano il 'futuro previsto' della risposta.

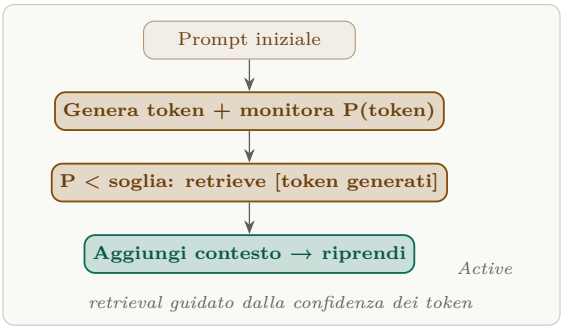
COME FUNZIONA

Il modello genera la risposta incrementalmente. Ad ogni step si monitora la probabilità dei token generati. Se la probabilità scende sotto una soglia (es. 0.5), i token a bassa confidenza vengono usati come query per un retrieval. Il contesto recuperato viene aggiunto al prompt e la generazione riprende da quel punto con l'informazione aggiornata.

PASSI IMPLEMENTATIVI

1. Inizia la generazione della risposta token per token
2. Monitora le probabilità dei token generati
3. Se $P(token) < threshold$: fermati
4. Usa i token generati come query forward-looking
5. Retrieve contesto rilevante per quei token
6. Aggiungi il contesto al prompt e riprendi la generazione

DIAGRAMMA ARCHITETTURALE



PRO

- + Retrieval solo quando necessario (efficiente)
- + Riduce le allucinazioni in modo chirurgico
- + Adattivo: si auto-regola sulla complessità della risposta

CONTRO

- – Richiede accesso alle probabilità dei token (non disponibile con tutte le API)
- – Implementazione complessa
- – Latenza variabile e imprevedibile

TOOL E LIBRERIE

LLM con accesso ai logprobs (GPT-4, Llama) • Any retriever • LangChain FLARE chain

COMPLESSITÀ ★★★ Alta

USE CASE

Generazione di testi lunghi su fatti, report automatici, riassunti di ricerca

ESEMPIO PYTHON

```
pip install langchain, openai
```

```python
from langchain.chains import FlareChain
from langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(model="gpt-4",
                 logprobs=True,  # necessario per FLARE
                 top_logprobs=1)

# FLARE: genera e monitora probabilita' token
flare_chain = FlareChain.from_llm(
    llm=llm,
    retriever=retriever,
    max_generation_len=256,
    min_prob=0.5,  # soglia di confidenza
)
# Se P(token) < 0.5 -> retrieve e reinserisce contesto
response = flare_chain.run(
    "Descrivi i progressi recenti in RAG multimodale")
```

RIFERIMENTI BIBLIOGRAFICI

- Jiang et al., *FLARE* (2023). arxiv:2305.06983

#32 Agentic RAG

Advanced Architecture Agentic / Orchestrated RAG

Key feature: **Tool-using agent + reranking + grounded generation**

DESCRIZIONE

Agentic RAG sostituisce la pipeline RAG fissa con un agent LLM che sceglie autonomamente quale tool di retrieval usare, in quale ordine e quante volte. L'agent può decidere di recuperare chunk, documenti completi, eseguire ricerche web, chiamare API esterne o combinare più strategie. Ogni tool restituisce evidenze che l'agent valuta prima di procedere. È l'architettura più flessibile ma anche la più complessa.

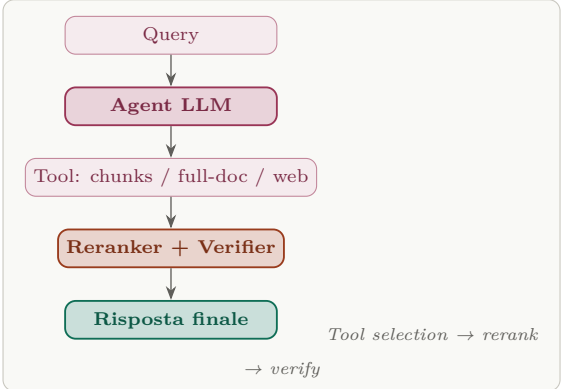
COME FUNZIONA

Un framework agent (LangGraph, PydanticAI) orchestra un LLM con accesso a tool di retrieval. L'LLM riceve la query, ragiona sui tool disponibili, li chiama nella sequenza ottimale, valuta i risultati e decide quando ha evidenze sufficienti per rispondere. Ogni tool call è verificata e i risultati vengono reranked prima della generazione.

PASSI IMPLEMENTATIVI

1. L'agent riceve la query e analizza i tool disponibili
2. Seleziona il tool: chunk search, full-doc, web search, API
3. Esegue il tool e valuta i risultati
4. Decide se i risultati sono sufficienti o se serve un altro tool
5. Opzionale: rerank dei risultati aggregati
6. Verifica il grounding della risposta prima di restituirla

DIAGRAMMA ARCHITETTURALE



PRO

- + Gestisce domande multi-step e complesse
- + Retrieval flessibile: sceglie autonomamente la strategia
- + Hook di verifica a ogni step

CONTRO

- – Latenza e costo elevati (N LLM call)
- – Difficile da debuggare senza buon tracing
- – Richiede guardrail robusti per evitare loop

TOOL E LIBRERIE

PydanticAI / LangGraph • Doc parsers (Docling, unstructured) • Vector DB + Reranker + Web search

COMPLESSITÀ ★★★ Alta

USE CASE

Enterprise doc QA, customer support con knowledge base complesse, research assistants

ESEMPIO PYTHON

```
pip install langgraph, langchain
```

```python
from langgraph.graph import StateGraph, END
from langchain_core.tools import tool

@tool
def search_docs(query: str) -> str:
    'Ricerca nei documenti aziendali.'
    return retriever.get_relevant_documents(query)

@tool
def search_web(query: str) -> str:
    'Ricerca web per info recenti.'
    return web_search(query)

# Agent con tool selection autonoma
from langchain.agents import create_tool_calling_agent
agent = create_tool_calling_agent(
    llm=llm,
    tools=[search_docs, search_web],
    prompt=agent_prompt
)
result = agent.invoke({"input": user_query})
```

RIFERIMENTI BIBLIOGRAFICI

- LangGraph documentation. langchain-ai.github.io
- Anthropic Tool Use docs. docs.anthropic.com

#33 Self-RAG

Advanced Architecture Self-critique + retrieval decision

Key feature: **LLM decide se/quando recuperare e verifica il grounding**

DESCRIZIONE

Self-RAG addestra un LLM a generare token speciali di riflessione che controllano il processo di retrieval e valutazione. Il modello impara a decidere autonomamente: (1) se il retrieval è necessario per questa query, (2) quali documenti recuperati sono rilevanti, (3) se la risposta generata è supportata dal contesto, (4) se la risposta è utile. Questo rende il sistema adattivo senza overhead fisso di LLM call esterne.

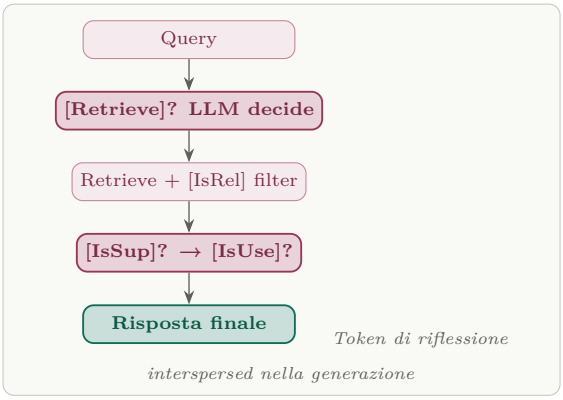
COME FUNZIONA

Il modello Self-RAG è fine-tuned con token speciali: [Retrieve] (decidi se recuperare), [IsRel] (questo documento è rilevante?), [IsSup] (la risposta è supportata?), [IsUse] (la risposta è utile?). A runtime, il modello genera questi token di riflessione interspersed nella generazione, guidando il processo senza richiedere LLM separati.

PASSI IMPLEMENTATIVI

1. LLM genera: [Retrieve] token se la query richiede evidenze esterne
2. Se [Retrieve]=Yes: retrieve k documenti
3. LLM valuta ogni documento: [IsRel]=Yes/No
4. Genera draft usando solo i documenti rilevanti
5. LLM valuta: [IsSup] (risposta supportata dal contesto?)
6. Se non supportata: rewrite query e re-retrieve
7. LLM valuta: [IsUse] → restituisce risposta finale

DIAGRAMMA ARCHITETTURALE



PRO

- + Evita retrieval non necessari (efficiente)
- + Tutto in un unico modello (no LLM call esterne)
- + Intercepta supporto debole prima della risposta

CONTRO

- – Richiede fine-tuning del modello (non plug-and-play)
- – Self-check dipende dalla qualità del fine-tuning
- – Non disponibile come API (serve modello custom)

TOOL E LIBRERIE

Self-RAG model (Asai et al., HuggingFace) • Retrievers standard • Vllm per inferenza efficiente

COMPLESSITÀ ★★★ Alta

USE CASE

Workload misto (alcune query non richiedono retrieval), sistemi sensibili alle allucinazioni

ESEMPIO PYTHON

pip install transformers, vllm

```python
# Self-RAG: usa il modello self-rag-llama3-8b
from vllm import LLM, SamplingParams

model = LLM("selfrag/selfrag_llama2_7b",
            max_model_len=4096)
params = SamplingParams(temperature=0,
                        max_tokens=512,
                        skip_special_tokens=False)

# Il modello genera token speciali di riflessione
output = model.generate(
    [f"### Instruction:\n{query}\n### Response:\n"],
    params)[0].outputs[0].text

# Parsing dei token speciali
if "[Retrieve]" in output:
    docs    = retriever.get_relevant_documents(query)
    context = "\n".join(d.page_content for d in docs)
    # Rigenera con contesto
```

RIFERIMENTI BIBLIOGRAFICI

- Asai et al., *Self-RAG* (2023). arxiv:2310.11511
- Self-RAG GitHub. github.com/AkariAsai/self-rag

#34 Corrective RAG (CRAG)

Advanced Architecture Corrective retrieval + fallback

Key feature: **Score retrieval → rewrite → fallback su fonti alternative**

DESCRIZIONE

CRAG affronta il problema del retrieval di bassa qualità aggiungendo un valutatore della qualità del retrieval stesso. Se il contesto recuperato ottiene uno score basso (sotto una soglia), CRAG attiva una strategia correttiva: riscrive la query e/o ricorre a fonti alternative (un secondo indice, web search). La correzione è trasparente all'utente finale ma migliora significativamente la robustezza del sistema.

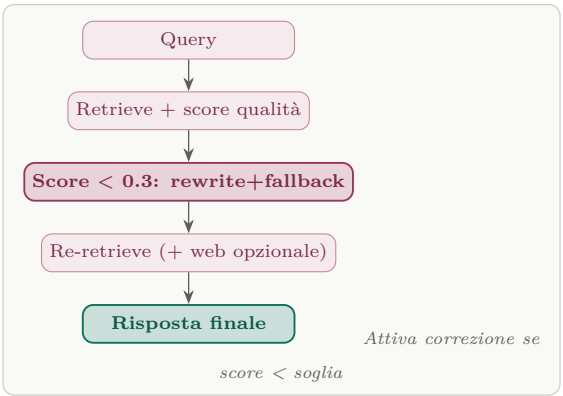
COME FUNZIONA

Dopo ogni retrieval, un LLM valuta la qualità del contesto su una scala 0–1. Se il score supera la soglia (0.7), si procede normalmente. Se è sotto (0.3), si attiva la correzione completa (rewrite + fallback). Nella zona grigia (0.3–0.7), si usa una combinazione di contesto originale e corretto.

PASSI IMPLEMENTATIVI

1. Retrieve $k = 8$ chunk con strategia standard
2. LLM evaluator: score qualità retrieval (0–1)
3. Se score > 0.7: usa il contesto e genera
4. Se score < 0.3: rewrite query + fallback (secondo indice o web)
5. Se $0.3 \leq$ score ≤ 0.7: merge contesto originale + corretto
6. Genera risposta sul contesto (eventualmente corretto)

DIAGRAMMA ARCHITETTURALE



PRO

- + Recovery automatico da retrieval scarso
- + Robusto su KB incomplete o datate
- + Fallback a web search amplia la copertura

CONTRO

- – Più componenti da mantenere
- – Fallback web può essere limitato o a pagamento
- – Attribuzione più complessa con contesti misti

TOOL E LIBRERIE

Retrieval evaluator (LLM) • Query rewriter • Web search API (opzionale, fallback)

COMPLESSITÀ ★★★ Alta

USE CASE

Support con KB incompleta, sistemi su dominio in rapida evoluzione, agenti di ricerca

ESEMPIO PYTHON

```
pip install langchain, openai
```

```python
# CRAG: valuta qualita' retrieval e corregge
from langchain_core.output_parsers import StrOutputParser

eval_prompt = (
    "Da 0 a 1, quanto e' rilevante questo contesto "
    "per la query? Rispondi solo con un numero. "
    "Query: {query}  Contesto: {context}"
)

def crag_retrieve(query):
    docs    = retriever.get_relevant_documents(query)
    context = "\n".join(d.page_content for d in docs)
    score   = float(llm.invoke(
        eval_prompt.format(query=query,
                           context=context)))
    if score < 0.3:
        # Fallback: rewrite + web search
        new_q = rewrite_query(query)
        docs  = web_retriever.get_relevant_documents(new_q)
    return docs, score
```

RIFERIMENTI BIBLIOGRAFICI

- Yan et al., *CRAG* (2024). arxiv:2401.15884

#35 Graph RAG

Advanced Architecture Graph-augmented retrieval

Key feature: **Entity extraction + graph traversal + vector search**

DESCRIZIONE

Graph RAG combina la ricerca vettoriale con la traversata di un knowledge graph. Le entità menzionate nella query vengono identificate tramite NER, i nodi corrispondenti vengono trovati nel grafo e i vicini (fino a 2 hop) vengono estratti. La ricerca vettoriale viene poi limitata ai chunk associati a questi nodi, migliorando la precisione per domande che richiedono la comprensione di relazioni tra entità.

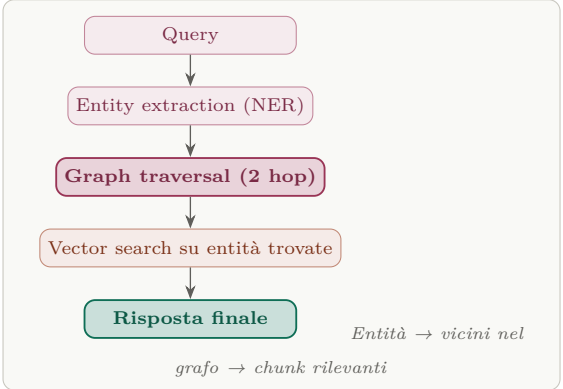
COME FUNZIONA

A index-time si costruisce un knowledge graph estraendo entità e relazioni dai documenti (NLP pipeline). Ogni chunk viene associato alle entità che menziona. A runtime, la query viene analizzata per estrarre entità chiave; il grafo viene traversato a partire da quelle entità per espandere il contesto relazionale; infine si esegue vector search limitata ai chunk delle entità trovate.

PASSI IMPLEMENTATIVI

1. Build: estrai entità e relazioni da tutti i documenti (NER + RE)
2. Build: costruisci knowledge graph (Neo4j, NetworkX)
3. Build: collega ogni chunk alle entità che menziona
4. Query: NER sulla query → entità seed
5. Traversa grafo: ottieni vicini a 2 hop
6. Vector search limitata ai chunk delle entità trovate
7. Genera risposta con contesto relazionale

DIAGRAMMA ARCHITETTURALE



PRO

- + Forte su domande multi-hop e relazionali
- + Coerenza tra entità attraverso i documenti
- + Riduce il context sprawl su domande specifiche

CONTRO

- – Knowledge graph richiede manutenzione
- – Qualità del entity linking critica
- – Infrastruttura aggiuntiva (Neo4j, etc.)

TOOL E LIBRERIE

Neo4j / NetworkX per il grafo • SpaCy / LLM per NER + RE • LangChain Graph RAG tutorial

COMPLESSITÀ ★★★ Alta

USE CASE

Knowledge base interconnessa, domande su relazioni tra entità, Q&A su dati strutturati

ESEMPIO PYTHON

```
pip install langchain, neo4j
```

```python
from langchain.graphs import Neo4jGraph
from langchain.chains import GraphCypherQAChain

graph = Neo4jGraph(url="bolt://localhost:7687",
                   username="neo4j", password="pwd")

# Popola il grafo da documenti
graph.query(
    "MERGE (e:Entity {name: entity_name}) "
    "MERGE (d:Document {id: doc_id_val}) "
    "MERGE (e)-[:MENTIONED_IN]->(d)",
    params={"name": entity, "doc_id": doc_id})

# Query RAG via Cypher auto-generato
chain = GraphCypherQAChain.from_llm(
    llm=llm, graph=graph,
    verbose=True, return_intermediate_steps=True)
result = chain.invoke({"query": user_query})
```

RIFERIMENTI BIBLIOGRAFICI

- Edge et al., *From Local to Global: GraphRAG* (2024). arxiv:2404.16130
- LangChain Graph RAG. python.langchain.com

#36 Microsoft GraphRAG

Advanced Architecture Community + graph summarization

Key feature: **Community summaries + local retrieval per domande globali**

DESCRIZIONE

Microsoft GraphRAG estende il Graph RAG con una fase di pre-elaborazione che costruisce community summaries riutilizzabili. Il corpus viene analizzato per estrarre entità e relazioni, organizzate in community (cluster di entità connesse). Per ogni community si genera un summary. Le domande globali ('Quali sono i temi principali?') vengono risposta usando i community summaries; le domande locali usano il retrieval standard.

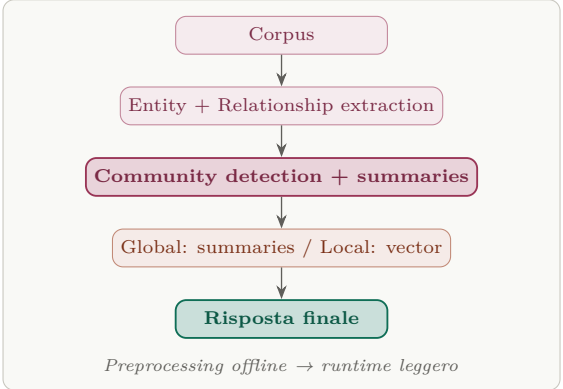
COME FUNZIONA

In una fase offline, si esegue entity extraction, relationship detection e community detection (Leiden algorithm) su tutto il corpus. Per ogni community si genera un summary con un LLM. A runtime, per domande globali si usano i community summaries; per domande locali si usa vector search con context da entità e relazioni. I due approcci possono essere combinati.

PASSI IMPLEMENTATIVI

1. Offline: entity extraction + relationship detection su tutto il corpus
2. Offline: community detection (Leiden algorithm)
3. Offline: genera community summary per ogni cluster
4. Runtime: classifica la query (globale vs locale)
5. Globale: retrieval su community summaries
6. Locale: vector search + graph context
7. Genera risposta con il contesto appropriato

DIAGRAMMA ARCHITETTURALE



PRO

- + Ottimo per domande corpus-wide e tematiche
- + Community summaries riutilizzabili (nessun costo a runtime)
- + Combina prospettiva globale e locale

CONTRO

- – Preprocessing molto pesante (ore su corpus grandi)
- – Possibili errori di astrazione nelle community summaries
- – Richiede tuning del community detection

TOOL E LIBRERIE

Microsoft GraphRAG (pip install graphrag) • Entity extraction pipeline • Leiden algorithm per community detection

COMPLESSITÀ ★★★ Alta

USE CASE

Large corpus aziendale, domande tematiche globali, analisi di trend su grandi archivi documentali

ESEMPIO PYTHON

pip install graphrag (microsoft)

```python
# Microsoft GraphRAG
# pip install graphrag

import asyncio
from graphrag.query.structured_search.global_search\
    .search import GlobalSearch
from graphrag.query.structured_search.local_search\
    .search import LocalSearch

# Global search: usa community summaries
global_search = GlobalSearch(
    llm=llm, context_builder=global_ctx_builder,
    token_encoder=token_encoder)

result = asyncio.run(
    global_search.asearch(
        "Quali sono i temi principali del corpus?"))
print(result.response)
# Local search per query specifiche
```

RIFERIMENTI BIBLIOGRAFICI

- Edge et al., *From Local to Global: GraphRAG* (2024). arxiv:2404.16130
- Microsoft GraphRAG GitHub. github.com/microsoft/graphrag

#37 RAPTOR

Advanced Architecture Tree-organized retrieval

Key feature: **Recursive summary tree per retrieval multi-livello**

DESCRIZIONE

RAPTOR costruisce un albero gerarchico di summary ricorsivi del corpus. I chunk foglia vengono clusterizzati e riassunti a formare nodi di livello 1; i nodi di livello 1 vengono clusterizzati e riassunti a formare nodi di livello 2, e così via fino alla radice. A retrieval-time, la query può essere soddisfatta sia dai summary ad alto livello (visione d'insieme) che dai chunk foglia (dettaglio), attraversando l'albero dall'alto verso il basso.

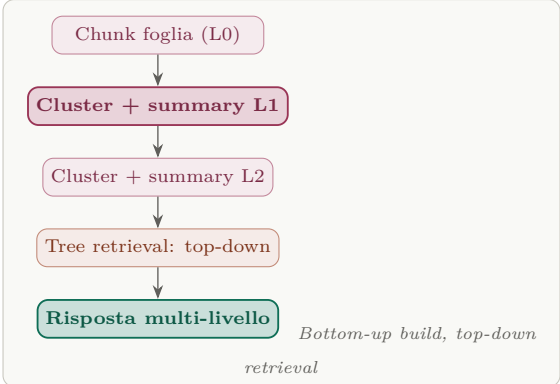
COME FUNZIONA

A build-time: si parte dai chunk foglia, si calcolano gli embedding, si clusterizza (GMM), si genera un summary per ogni cluster. Il processo si ripete sui summary fino alla radice. A runtime: la query viene usata per cercare sia nella radice (overview) che nelle foglie (dettaglio) attraverso la struttura ad albero.

PASSI IMPLEMENTATIVI

1. Build: chunking base del corpus (foglie)
2. Build: embedding + GMM clustering dei chunk foglia
3. Build: genera summary per ogni cluster (nodi L1)
4. Build: ripeti su L1 → L2 → ... → radice
5. Runtime: query search nell'albero (radice + foglie)
6. Runtime: drill-down dall'overview ai dettagli specifici
7. Genera risposta con contesto multi-livello

DIAGRAMMA ARCHITETTURALE



PRO

- + Gestisce efficacemente documenti molto lunghi
- + Permette overview di alto livello e dettaglio specifico
- + Migliore copertura rispetto a flat retrieval

CONTRO

- – Costo build elevato (summary ricorsivi LLM)
- – Errori nei summary si propagano verso l'alto
- – Complessità architetturale significativa

TOOL E LIBRERIE

Recursive LLM summarizer • GMM clustering (scikit-learn) • LlamaIndex RAPTOR implementation

COMPLESSITÀ ★★★ Alta

USE CASE

Report lunghi, paper scientifici, corpora con struttura narrativa complessa

ESEMPIO PYTHON

```
pip install llama-index
```

```python
from llama_index.packs.raptor import RaptorPack

# RAPTOR: costruisce albero di summary ricorsivi
raptor_pack = RaptorPack(
    documents,
    embed_model=OpenAIEmbedding(
        model="text-embedding-3-small"),
    llm=OpenAI(model="gpt-4o-mini"),
    vector_store=vector_store,
    similarity_top_k=2,
    mode="collapsed",   # o "tree_traversal"
    transformations=[
        SentenceSplitter(chunk_size=1024)],
)
# Retrieval che traversa l'albero
query_engine = raptor_pack.get_modules()[
    "query_engine"]
response = query_engine.query(user_query)
```

RIFERIMENTI BIBLIOGRAFICI

- Sarthi et al., *RAPTOR* (2024). arxiv:2401.18059
- LlamaIndex RAPTOR pack. docs.llamaindex.ai

#38 Sophisticated Controllable Agent

Advanced Architecture Deterministic agentic RAG

Key feature: **Control graph deterministico + verifier + re-planning**

DESCRIZIONE

A differenza dell'Agentic RAG free-form, il Sophisticated Controllable Agent usa un grafo di controllo deterministico: la query viene decomposta in sub-task con query di retrieval esplicite e pre-definite. Ogni sub-task viene eseguito in sequenza, verificato da un verifier LLM, e se la verifica fallisce il sistema ri-pianifica quel sub-task. Il risultato finale è la sintesi di tutti i sub-risultati verificati. Questo pattern è preferito in contesti enterprise dove l'auditabilità è fondamentale.

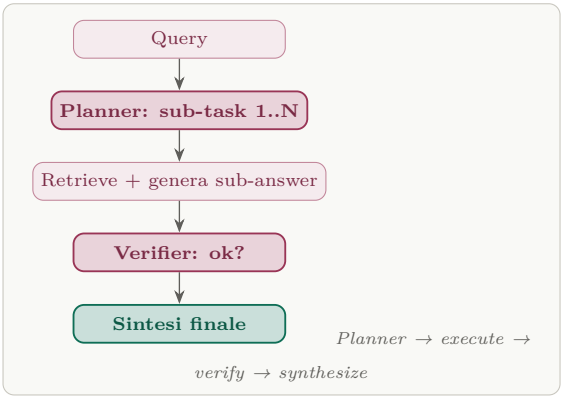
COME FUNZIONA

Un planner LLM decompone la query in una lista ordinata di sub-task, ognuno con una query di retrieval specifica. Ogni sub-task viene eseguito: retrieve → genera → verifica. Se il verifier rifiuta la risposta al sub-task, si esegue un rewrite della retrieval query e si riprova. La sintesi finale integra tutti i sub-risultati verificati.

PASSI IMPLEMENTATIVI

1. Planner LLM: decomponi query in sub-task con retrieval query esplicite
2. Per ogni sub-task: retrieve con la query specifica
3. Genera risposta al sub-task con il contesto recuperato
4. Verifier LLM: la risposta al sub-task è supportata?
5. Se no: rewrite retrieval query e riprova (max 2)
6. Accumula sub-risultati verificati
7. Sintesi finale: LLM integra tutti i sub-risultati

DIAGRAMMA ARCHITETTURALE



PRO

- + Più prevedibile e auditabile degli agenti free-form
- + Step verificabili e tracciabili
- + Forte su task multi-fase complessi

CONTRO

- – Design effort elevato per il planner e le strategie
- – Latenza e costo alti (N sub-task × retry)
- – Verifier imperfetti possono bloccare step corretti

TOOL E LIBRERIE

LangGraph per FSM • Custom planner LLM • Vector DB + reranker per ogni sub-task

COMPLESSITÀ ★★★ Alta

USE CASE

Workflow enterprise riproducibili, compliance, generazione di report strutturati

ESEMPIO PYTHON

```
pip install langgraph, langchain
```

```python
from langgraph.graph import StateGraph
from typing import TypedDict, List

class AgentState(TypedDict):
    query: str
    subtasks: List[str]
    results:  List[str]
    verified: bool

def planner(state):
    subtasks = llm.invoke(
        f"Decomponi in sub-task: {state['query']}")
    return {"subtasks": subtasks.split("\n")}

def executor(state):
    results = []
    for task in state["subtasks"]:
        docs   = retriever.get_relevant_documents(task)
        answer = qa_chain.run(task, docs)
        ok     = verifier_llm.invoke(
            f"La risposta e' grounded? {answer}")
        results.append(answer if "SI" in ok else "")
    return {"results": results, "verified": True}

graph = StateGraph(AgentState)
graph.add_node("plan", planner)
graph.add_node("exec", executor)
```

RIFERIMENTI BIBLIOGRAFICI

- LangGraph docs. langchain-ai.github.io
- Yao et al., *ReAct* (2023). arxiv:2210.03629
- Nakano et al., *WebGPT* (2022). arxiv:2112.09332

#39 DeepEval

Evaluation RAG evaluation framework

Key feature: **Testing harness CI-ready con metriche LLM-judge**

DESCRIZIONE

DeepEval è un framework open-source per la valutazione automatica di sistemi RAG. Definisce un set di metriche standardizzate: Correctness (la risposta è corretta rispetto al ground truth?), Faithfulness (la risposta è supportata dal contesto recuperato?), Answer Relevancy (la risposta è rilevante per la query?), Contextual Precision/Recall (il contesto recuperato è preciso e completo?). Le metriche usano LLM-as-judge o modelli specializzati.

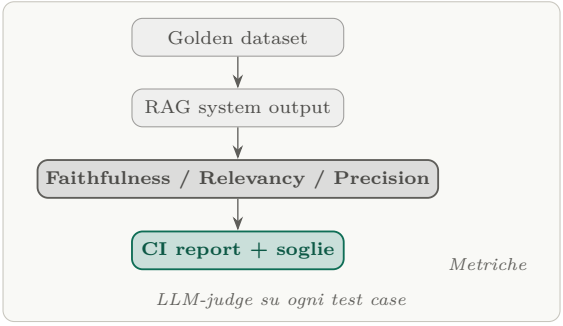
COME FUNZIONA

Si costruisce un golden dataset: per ogni test case si definisce query, contesto atteso e risposta attesa. Il sistema RAG viene eseguito su ogni test case. DeepEval calcola le metriche usando LLM-judge (GPT-4) o modelli specializzati. I risultati vengono aggregati in un report. Il framework si integra con pytest per il testing continuo in CI/CD.

PASSI IMPLEMENTATIVI

1. Costruisci golden dataset: {query, expected_context, expected_answer}
2. Esegui il sistema RAG su ogni test case
3. Calcola Faithfulness: risposta supportata dal contesto recuperato?
4. Calcola Answer Relevancy: risposta pertinente alla query?
5. Calcola Contextual Precision/Recall
6. Aggrega metriche in report
7. Integra con pytest + CI per regression testing

DIAGRAMMA ARCHITETTURALE



PRO

- + Metriche standardizzate e riproducibili
- + Protezione da regressioni
- + Integrazione CI/CD nativa con pytest

CONTRO

- – Richiede golden dataset di qualità (costoso da creare)
- – Rumore dal LLM-judge (non deterministico)
- – Costo se usa GPT-4 come judge

TOOL E LIBRERIE

DeepEval (pip install deepeval) • Golden datasets • pytest + GitHub Actions per CI

COMPLESSITÀ ★★☆ Media

USE CASE

Valutazione sistemi RAG in produzione, A/B testing di configurazioni, regression testing

ESEMPIO PYTHON

```
pip install deepeval
```

```python
from deepeval import evaluate
from deepeval.metrics import (
    FaithfulnessMetric, AnswerRelevancyMetric,
    ContextualPrecisionMetric,
    ContextualRecallMetric)
from deepeval.test_case import LLMTestCase
import pytest

metrics = [FaithfulnessMetric(threshold=0.7),
           AnswerRelevancyMetric(threshold=0.7),
           ContextualPrecisionMetric(threshold=0.6),
           ContextualRecallMetric(threshold=0.6)]

@pytest.mark.parametrize("case", golden_dataset)
def test_rag(case):
    tc = LLMTestCase(**case)
    evaluate([tc], metrics)  # integra con pytest CI
```

RIFERIMENTI BIBLIOGRAFICI

- Es et al., *RAGAS* (2023). arxiv:2309.15217
- DeepEval GitHub. github.com/confident-ai/deepeval

#40 Explainable Retrieval

Explainability Attribution / Transparency

Key feature: **Provenienza + score + span citati per ogni risposta**

DESCRIZIONE

Explainable Retrieval aggiunge un layer di trasparenza al sistema RAG: ogni risposta è accompagnata da: (1) i chunk sorgente con i loro similarity score, (2) una spiegazione di perché ogni chunk è stato recuperato (matched terms, metadata), (3) la parte specifica del chunk che supporta ogni affermazione della risposta. Questo è fondamentale per sistemi enterprise dove gli utenti devono poter verificare le fonti.

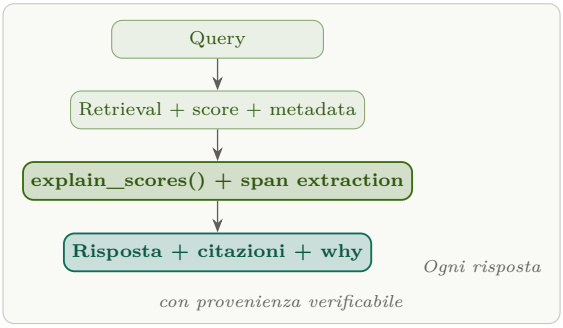
COME FUNZIONA

Dopo il retrieval, il sistema genera sia la risposta che le spiegazioni. Per ogni chunk recuperato si calcolano: similarity score, matched keywords, sezione del documento sorgente. La risposta viene generata con istruzioni esplicite di citazione. Le citazioni vengono rese cliccabili nell'interfaccia utente, permettendo all'utente di verificare direttamente le fonti.

PASSI IMPLEMENTATIVI

1. Retrieval con score e metadata espliciti (non solo chunk text)
2. Per ogni chunk: calcola matched keywords, path sezione, score
3. Genera risposta con citazioni: 'Secondo [fonte X, p.Y]: ...'
4. Evidenzia gli span specifici di ogni chunk che supportano ogni affermazione
5. Opzionale: tracing con OpenTelemetry per audit trail completo
6. UI: rendi le citazioni cliccabili e verificabili

DIAGRAMMA ARCHITETTURALE



PRO

+ Aumenta la fiducia degli utenti nelle risposte
+ Facilita il debugging e il miglioramento del sistema
+ Audit trail completo per compliance

CONTRO

– Lavoro UX significativo per rendere le citazioni navigabili
– Le spiegazioni possono aumentare la lunghezza delle risposte
– Metadata sensibili nei path potrebbero richiedere filtering

TOOL E LIBRERIE

Citation spans + metadata • OpenTelemetry + LangSmith • Langfuse per tracing e audit

COMPLESSITÀ ★★☆ Media

USE CASE

Enterprise con human-in-the-loop, sistemi legali/medici, assistenti dove la verifica è critica

ESEMPIO PYTHON

```
pip install langfuse, opentelemetry
```

```python
from langfuse import Langfuse
from langfuse.decorators import observe, langfuse_context

lf = Langfuse()

@observe()  # traccia automaticamente input/output
def rag_pipeline(query: str) -> dict:
    docs = retriever.get_relevant_documents(query)
    # Aggiungi metadata di provenienza
    context_with_meta = [
        {"text": d.page_content,
         "source": d.metadata["source"],
         "score": d.metadata.get("score", 0),
         "section": d.metadata.get("section", "")}
        for d in docs]
    answer = llm.invoke(
        build_prompt(query, context_with_meta))
    # Score manuale della risposta
    langfuse_context.score_current_trace(
        name="citation-quality", value=0.9)
    return {"answer": answer,
            "sources": context_with_meta}
```

RIFERIMENTI BIBLIOGRAFICI

- Guo et al., *Attributable RAG* (2024). arxiv:2402.04315
- OpenTelemetry for LLMs. opentelemetry.io
- Langfuse docs. langfuse.com